

claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf_b7edcaa8-69d\agent-af0b03c7b40337df6.jsonl`  
- SHA-256: `a77b5b706640ba1d3cceb2d447e30478f247d4e8a1f68e662576358e35f1bc7a`  
- Source modified: `2026-06-21T14:39:44+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_b7edcaa8-69d`  
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`
```

Transcript

1. user (2026-06-21T14:33:51.170Z)

Spec the **runnable-tool North Star** = make the UI shareable with alpha users (NOT GPU-gated, NOT M12). Three tickets, T4 ? T2 ? T11. Read in C:/Users/avidu/Projects/khelsutra: web/src/components/IngestPanel.tsx + web/src/api/client.ts + web/src/App.tsx (the current gs://-URI ingest), docs/LOCAL_APPLIANCE_ARCHITECTURE.md (T2/T4/T11 + \$4 topology + \$10 owner Qs), docs/PORTABILITY.md + site/scripts/provision-vm.sh + teardown-vm.sh (the droplet provision/teardown that exists). Read in C:/Users/avidu/Projects/badminton-highlight-indexer: backend/api/uploads.py (the EXISTING chunked-upload endpoints /api/upload/init|part|complete|DELETE), backend/main.py (/api/process + the static-serve), backend/config (CORS/bind), deploy notes. Produce THREE tickets:

- T4 (in-UI local file UPLOAD ? the keystone): wire the SPA to the EXISTING chunked-upload endpoints (init/part/complete) + a file-picker UI in/alongside IngestPanel, then hand the resulting server path to the existing /api/process flow. Specify the client additions (api/client.ts upload helpers), the IngestPanel/new-component UI, progress, max-part size, and how it joins the existing poll?load?compile?download path. Net-new vs reused.
- T2 (droplet appliance shell ? hosted + shareable): serve the built SPA + reverse-proxy /api ? the engine on the CPU droplet (168.144.126.176) behind Cloudflare Access (edge auth), engine on Gemini/CPU (NO GPU). Specify: build the SPA, the reverse-proxy/static-host config (Caddy or the existing Node server), wiring /api to 127.0.0.1:8000, the Cloudflare Access setup at a high level, engine deploy (the #281/#289-fixed engine + GEMINI_API_KEY + google-cloud-storage), key rotation, and which PORTABILITY/provision scripts already help. What's reused vs net-new; owner steps.
- T11 (retention): the output/ + uploads/ sweep before non-owner exposure.

For each: filesToTouch, changeSpec, newI18nKeys, testPlan, gotchas, effort, gated. Return structured JSON (tickets:[T4,T2,T11]).

2. assistant (2026-06-21T14:33:53.719Z)

I'll research the three repos to spec these tickets concretely. Let me start by pulling the relevant repos and reading the key files.

3. user (2026-06-21T14:33:58.933Z)

```
warning: fetch updated the current branch head.  
fast-forwarding your working tree from  
commit a0475bc0704479d5f72e7fe0b0b98aef5836c12d.  
error: The following untracked working tree files would be overwritten by merge:  
    docs/MULTILINGUAL_EXPANSION.md  
Please move or remove them before you merge.  
Aborting  
fatal: Cannot fast-forward your working tree.  
After making sure that you saved anything precious from  
$ git diff a0475bc0704479d5f72e7fe0b0b98aef5836c12d  
output, run  
$ git reset --hard
```

to recover.
---BHI---
Already up to date.

4. user (2026-06-21T14:33:58.946Z)

```
1      // The front-half ingest screen (appliance S1/S2, tracker B-P2). Point the tool at a
video ? a
2      // `gs://` bucket URI or a path on the engine host ? and it submits to POST
/api/process, shows
3      // live progress while polling GET /api/jobs/{id}, and hands the resulting video_id to
the existing
4      // rally-selection back-half (App lifts ?video=<id>). Honest by construction: the engine
reports
5      // coarse phase WORDS (not a %), and a success-shaped failure (done but no rallies) is
shown, never
6      // silently treated as a load.
7      import { useState } from "react";
8      import type { FormEvent } from "react";
9      import { useTranslation } from "react-i18next";
10     import { useIngestJob } from "../hooks/useIngestJob";
11
12     interface Props {
13       /** Called with the engine's video_id once a submitted job finishes successfully. */
14       onLoaded: (videoId: string) => void;
15     }
16
17     export function IngestPanel({ onLoaded }: Props) {
18       const { t } = useTranslation();
19       const [uri, setUri] = useState("");
20       const { phase, submit, cancel, reset, busy } = useIngestJob(onLoaded);
21
22       const onSubmit = (e: FormEvent) => {
23         e.preventDefault();
24         submit(uri);
25       };
26
27       // `engine` failures carry the engine's own message (a 400 scheme/path detail, a
worker error);
28       // every other code maps to a friendly localized line ? the network case especially,
since this
29       // is the first screen a fresh user hits and "is the engine running?" must read
plainly.
30       const errorMsg =
31         phase.status === "error"
32           ? phase.error.code === "engine"
33             ? (phase.error.detail ?? t("ingest.failed", { error: "" })))
34             : t(`ingest.${phase.error.code}`)
35           : null;
36
37       return (
38         <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
39           <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
40           <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
41
42           <form onSubmit={onSubmit} className="mt-3 flex flex-wrap items-center gap-2">
43             <input
44               type="text"
45               value={uri}
46               onChange={(e) => setUri(e.target.value)}
47               placeholder={t("ingest.uriPlaceholder")}
48               aria-label={t("ingest.uriAriaLabel")}
49               disabled={busy}
50               className="min-w-[18rem] flex-1 rounded-xl border border-black/15 bg-white
```

```

px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
51         />
52         <button
53             type="submit"
54             disabled={busy}
55             className="rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-white
disabled:opacity-40"
56         >
57             {phase.status === "submitting" ? t("ingest.submitting") : t("ingest.submit")}
58         </button>
59         {busy && (
60             <button
61                 type="button"
62                 onClick={cancel}
63                 className="rounded-xl border border-black/15 px-4 py-2 text-sm hover:border-
green"
64             >
65                 {t("ingest.cancel")}
66             </button>
67         )}
68     </form>
69
70     <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
71
72     {/* Live status (aria-live so screen readers hear progress). Indeterminate: the
engine reports
73         phase words ("hashing", "segmenting"), not a percentage ? so a spinner + the
phase word. */}
74     <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
75         {busy && (
76             <p className="flex items-center gap-2 text-ink/70">
77                 <Spinner />
78                 <span>
79                     {t("ingest.working")}
80                     {phase.status === "polling" && phase.progress
81                         ? ` · ${t("ingest.progress", { progress: phase.progress })}`
82                         : ""}
83                 </span>
84             </p>
85         )}
86         {phase.status === "error" && (
87             <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
88                 <p>{errorMsg}</p>
89                 <button
90                     type="button"
91                     onClick={reset}
92                     className="mt-2 rounded-lg border border-red-300 px-3 py-1 text-xs
hover:bg-red-100"
93                 >
94                     {t("ingest.retry")}
95                 </button>
96             </div>
97         )}
98     </div>
99 </section>
100 );
101 }
102
103 function Spinner() {
104     return (
105         <span
106             className="inline-block h-3.5 w-3.5 animate-spin rounded-full border-2 border-
ink/20 border-t-ink/70"
107             aria-hidden="true"

```

```

108     />
109   );
110 }
111

```

5. user (2026-06-21T14:33:58.946Z)

```

1   import { useEffect, useMemo, useState } from "react";
2   import { useTranslation, Trans } from "react-i18next";
3   import { useRallySelection } from "../hooks/useRallySelection";
4   import { RallyGrid } from "../components/RallyGrid";
5   import { QueryBar } from "../components/QueryBar";
6   import { SelectionFooter } from "../components/SelectionFooter";
7   import { LanguageSwitcher } from "../components/LanguageSwitcher";
8   import { CompileResult, type BuildState } from "../components/CompileResult";
9   import { IngestPanel } from "../components/IngestPanel";
10  import { runQuery, type ParsedQuery } from "../lib/query";
11  import { floorRisk } from "../lib/floor";
12  import { reelFilename } from "../lib/format";
13  import { parseDetail } from "../lib/errors";
14  import { log, newCorrelationId } from "../lib/log";
15  import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "../api/client";
16  import type { EngineConfig, RallySegment } from "../api/types";
17
18  /** One compact, structured description of how a query was understood (for the trace
log). */
19  function parseTrace(q: ParsedQuery) {
20    return {
21      raw: q.raw,
22      recognized: q.recognized,
23      filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
24      sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
25      limit: q.limit ?? null,
26      unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
27    };
28  }
29
30  // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
31  // contract; only honest fields are surfaced. A real session loads from POST /api/query.
32  const DEMO_RALLIES: RallySegment[] = [
33    { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
34    { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
35    { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
36    { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
37    { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
38    { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
39  ];
40
41  type LoadState =
42    | { status: "demo" }
43    | { status: "loading" }
44    | { status: "loaded"; count: number }
45    | { status: "error"; error: string };
46
47  export default function App() {
48    const { t } = useTranslation();
49    // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from

```

```

the URL,
50      // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
51      // without a reload (it also rewrites ?video= so a refresh is stable).
52      const [videoId, setVideoId] = useState<string | null>{
53        () => new URLSearchParams(window.location.search).get("video"),
54      };
55
56      const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
57      const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
58      const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
59      const [reelName, setReelName] = useState("highlights");
60      const [build, setBuild] = useState<BuildState>({ status: "idle" });
61
62      // Display order is separate from selection: a query can re-sort the cards while the
shared
63      // selection set (sel) stays the single source of truth for what lands in the reel.
64      const sel = useRallySelection(rallies, "all");
65
66      // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
67      useEffect(() => {
68        if (!videoId) return;
69        const cid = newCorrelationId();
70        log("info", "rallies.load_start", { cid, video_id: videoId });
71        // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
72        // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
73        setLoadState({ status: "loading" });
74        let alive = true;
75        listRallies(videoId, cid)
76          .then((rs) => {
77            if (!alive) return;
78            setRallies(rs);
79            setOrder(rs);
80            sel.apply(rs.map((r) => r.id));
81            setLoadState({ status: "loaded", count: rs.length });
82            log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
83          })
84          .catch((err) => {
85            if (!alive) return;
86            setLoadState({ status: "error", error: String(err) });
87            log("error", "rallies.load_failed", { cid, video_id: videoId, error: String(err)
});
88          });
89        return () => {
90          alive = false;
91        };
92        // videoId is stable for the page; sel.apply is a stable callback.
93        // eslint-disable-next-line react-hooks/exhaustive-deps
94      }, [videoId]);
95
96      // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
97      // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
98      const [cfg, setCfg] = useState<EngineConfig | null>(null);
99      useEffect(() => {
100        let alive = true;
101        getConfig()
102          .then((c) => alive && setCfg(c))
103          .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
104        return () => {
105          alive = false;
106        };

```

```

107     }, []);
108     const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
109
110     // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
111     const ordinalOf = useMemo(() => {
112         const m = new Map<number, number>();
113         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
114         return (id: number) => m.get(id) ?? 0;
115     }, [rallies]);
116
117     // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
118     const onBuild = async () => {
119         if (build.status === "building") return;
120         const cid = newCorrelationId();
121         const filename = reelFilename(reelName);
122         const ids = [...sel.selected];
123         if (!videoId) {
124             log("info", "build.no_video", { cid, selected: ids.length });
125             setBuild({
126                 status: "error",
127                 error: t("build.noVideo"),
128             });
129             return;
130         }
131         setBuild({ status: "building" });
132         log("info", "build.start", {
133             cid,
134             video_id: videoId,
135             filename,
136             selected: ids.length,
137             total_sec: Math.round(sel.totalSec),
138             below_floor: warning?.atRisk.length ?? 0,
139         });
140         try {
141             const res = await compileReel(videoId, ids, filename, cid);
142             const url = highlightUrl(videoId, filename);
143             setBuild({ status: "done", url, filename });
144             log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
145         } catch (err) {
146             const status = err instanceof ApiError ? err.status : undefined;
147             const detail = err instanceof ApiError && err.body ? parseDetail(err.body) :
String(err);
148             setBuild({ status: "error", error: detail });
149             log("error", "build.failed", { cid, video_id: videoId, filename, status, error:
String(err) });
150         }
151     };
152
153     // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
154     // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
155     // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
156     const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
157         const cid = newCorrelationId();
158         log("info", "query.parse", { cid, source, ...parseTrace(q) });
159
160         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
161         // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
162         if (q.unavailable.length > 0) {

```

```

163      log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
164    }
165
166    if (!q.recognized) {
167      log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
168      return; // empty / unparseable ? leave the user's curation untouched
169    }
170
171    try {
172      const { ordered, selectedIds } = runQuery(rallies, q);
173      setOrder(ordered);
174      sel.apply(selectedIds);
175      log("info", "query.apply", {
176        cid,
177        source,
178        filters: q.filters.length,
179        sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
180        limit: q.limit ?? null,
181        total: rallies.length,
182        selected: selectedIds.length,
183        selected_ids: selectedIds,
184        dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
185      });
186      if (selectedIds.length === 0) {
187        // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
188        log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
189      }
190    } catch (err) {
191      // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
192      log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
193    }
194  };
195
196  // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
197  const onToggle = (id: number) => {
198    log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
199    sel.toggle(id);
200  };
201  const onBulk = (op: "select_all" | "clear" | "invert") => {
202    const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
203    log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
204    if (op === "select_all") sel.selectAll();
205    else if (op === "clear") sel.clearAll();
206    else sel.invert();
207  };
208
209  return (
210    <main className="min-h-screen bg-paper text-ink pb-24">
211      <header className="bg-ink text-white">
212        <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
213          <span className="text-xl font-extrabold tracking-tight">
214            Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
215          </span>
216          <LanguageSwitcher />
217        </div>

```

```

218         </header>
219
220         <section className="mx-auto max-w-5xl px-6 py-10">
221             <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
222                 {t("app.badge")}
223             </span>
224             <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
225             <p className="mt-2 max-w-2xl text-ink/70">
226                 <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
227             </p>
228
229             {/* Front-half ingest (B-P2): when no video is loaded, point the tool at a
bucket URI / host
230                 path ? process ? load. Once a video_id exists, the existing back-half takes
over. */}
231             {!videoId && (
232                 <IngestPanel
233                     onLoad={id => {
234                         setVideoId(id);
235                         window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
236                     }}
237                 />
238             )}
239
240             <ModeBanner load={loadState} videoId={videoId} />
241
242             <div className="mt-6">
243                 <QueryBar onApply={onApplyQuery} />
244             </div>
245
246             <div className="mt-6 flex flex-wrap gap-2 text-sm">
247                 <button type="button" onClick={() => onBulk("select_all")} className="rounded-
full border border-black/15 px-3 py-1 hover:border-green">{t("bulk.selectAll")}</button>
248                 <button type="button" onClick={() => onBulk("clear")} className="rounded-full
border border-black/15 px-3 py-1 hover:border-green">{t("bulk.clear")}</button>
249                 <button type="button" onClick={() => onBulk("invert")} className="rounded-full
border border-black/15 px-3 py-1 hover:border-green">{t("bulk.invert")}</button>
250             </div>
251
252             <div className="mt-5">
253                 <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
254             </div>
255
256             <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
257         </section>
258
259         <SelectionFooter
260             count={sel.count}
261             totalSec={sel.totalSec}
262             reelName={reelName}
263             onReelNameChange={setReelName}
264             onBuild={onBuild}
265             building={build.status === "building"}
266             warning={warning}
267         />
268     </main>
269 );
270 }
271
272 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
273     if (load.status === "demo") {
274         return (

```



```

275         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
276             <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
277         </p>
278     );
279 }
280 if (load.status === "loading") {
281     return (
282         <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
283             <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
284         </p>
285     );
286 }
287 if (load.status === "error") {
288     return (
289         <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
290 text-red-800" data-testid="mode-banner">
291             <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
292 components={{ code: <code /> }} />
293         </p>
294     );
295 }
296 return (
297     <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
298         <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
299 <strong className="text-ink/70" /> }} />
300     </p>
301 );
302 }
303 }
304 }

```

6. user (2026-06-21T14:33:58.956Z)

```

1 // Typed client for the engine FastAPI. SAME-ORIGIN relative `/api/*` (the appliance
2 // UI + engine behind one origin; in dev Vite proxies `/api` ? :8000). Every call
3 // correlation id (header + logs) so a journey is traceable end to end.
4 import { log, newCorrelationId } from "../lib/log";
5 import type { RallySegment, QueryResponse, CompileResponse, EngineConfig,
6 ProcessResponse, JobStatus } from "./types";
7
8 const BASE = "/api";
9
10 export class ApiError extends Error {
11     readonly cid?: string;
12     readonly status?: number;
13     readonly body?: string;
14     constructor(message: string, opts: { cid?: string; status?: number; body?: string;
15 cause?: unknown } = {}) {
16         super(message);
17         this.name = "ApiError";
18         this.cid = opts.cid;
19         this.status = opts.status;
20         this.body = opts.body;
21         if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
22     }
23 }
24
25 interface RequestOptions {
26     method?: string;
27     body?: string;
28     headers?: Record<string, string>;
29     /** Correlation id to thread through this call; auto-generated if absent. */
30     cid?: string;
31 }

```

```

29     }
30
31     async function request<T>(path: string, opts: RequestOptions = {}): Promise<T> {
32         const cid = opts.cid ?? newCorrelationId();
33         const url = `${BASE}${path}`;
34         const method = opts.method ?? "GET";
35         log("info", "api.request", { cid, method, url });
36
37         let res: Response;
38         try {
39             res = await fetch(url, {
40                 method,
41                 body: opts.body,
42                 headers: { "content-type": "application/json", "x-correlation-id": cid,
... (opts.headers ?? {}) },
43             });
44         } catch (err) {
45             // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
46             log("error", "api.network_error", { cid, method, url, error: String(err) });
47             throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
48         }
49
50         if (!res.ok) {
51             const body = await res.text().catch(() => "");
52             log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
53             throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
54         }
55
56         const data = (await res.json()) as T;
57         log("info", "api.response", { cid, method, url, status: res.status });
58         return data;
59     }
60
61     /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
62     * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
63     * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
64     export async function processVideo(videoPath: string, cid?: string):
Promise<ProcessResponse> {
65         return request<ProcessResponse>("/process", {
66             method: "POST",
67             body: JSON.stringify({ video_path: videoPath }),
68             cid,
69         });
70     }
71
72     /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
73     export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
74         return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
75     }
76
77     /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
78     export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
79         const data = await request<QueryResponse>("/query", {
80             method: "POST",
81             body: JSON.stringify({ video_id: videoId }),
82             cid,
83         });

```

```

84     return data.play_segments ?? [];
85 }
86
87 /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */
88 export async function compileReel(
89     videoId: string,
90     segmentIds: number[],
91     outputFilename: string,
92     cid?: string,
93 ): Promise<CompileResponse> {
94     return request<CompileResponse>("/compile", {
95         method: "POST",
96         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename }),
97         cid,
98     });
99 }
100
101 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
102 export function highlightUrl(videoId: string, filename: string): string {
103     return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
104 }
105
106 /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
107 export async function getConfig(cid?: string): Promise<EngineConfig> {
108     return request<EngineConfig>("/config", { method: "GET", cid });
109 }
110
111 /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
112 export function minShotCount(cfg: EngineConfig): number | undefined {
113     return cfg.stitching?.min_shot_count;
114 }
115

```

7. assistant (2026-06-21T14:34:02.169Z)

Now let me read the BHI uploads endpoints, main.py, config, and the khelsutra architecture docs.

8. user (2026-06-21T14:34:03.141Z)

```

1     """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3     Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4     proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5     under security.upload_dir, then submitted to /api/process by the returned path.
6     Upload state is in-process by design (this is the single-box alpha): a server restart
7     aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8     arrives with PR-3 identity scoping.
9
10    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12    reads the live value the server set at startup (and so tests that monkeypatch
13    ``backend.main.global_config`` flow through unchanged).
14    """
15    import os
16    import threading
17    import uuid
18    from typing import Any, Dict, Optional
19
20    from fastapi import APIRouter, HTTPException, Request
21    from pydantic import BaseModel

```

```

22
23 from backend.utils.hashing import compute_video_id # canonical file-identity hashing
24 from backend.utils.paths import safe_output_filename
25
26 router = APIRouter()
27
28 _uploads: Dict[str, Dict[str, Any]] = {}
29 _uploads_lock = threading.Lock()
30
31
32 class UploadInitRequest(BaseModel):
33     filename: str
34     total_size_bytes: Optional[int] = None
35
36
37 class UploadCompleteRequest(BaseModel):
38     total_parts: int
39
40
41 def _upload_cfg():
42     # Resolved lazily (import inside the function) to read the LIVE startup config that
43     # main() sets on the module global, and to avoid a circular import at module load
44     # (main.py imports this router; this module must not import main at import time).
45     import backend.main as _main
46     sec = getattr(_main.global_config, "security", None)
47     upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
48     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
49     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
50     exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
None) or ["mp4", "mov"])]
51     return upload_dir, max_bytes, part_bytes, exts
52
53
54 def _abort_upload(upload_id: str) -> None:
55     with _uploads_lock:
56         st = _uploads.pop(upload_id, None)
57         if st:
58             try:
59                 os.remove(st["tmp_path"])
60             except OSError:
61                 pass
62
63
64 @router.post("/api/upload/init")
65 def upload_init(req: UploadInitRequest):
66     """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
67     upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
68     try:
69         name = safe_output_filename(req.filename)
70     except ValueError as e:
71         raise HTTPException(status_code=400, detail=str(e)) from e
72     ext = os.path.splitext(name)[1].lower().lstrip(".")
73     if ext not in exts:
74         raise HTTPException(status_code=400,
75                             detail=f"Extension '{ext}' not allowed. Allowed:
{sorted(exts)}")
76     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
77         raise HTTPException(status_code=413,
78                             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
79     os.makedirs(upload_dir, exist_ok=True)
80     upload_id = uuid.uuid4().hex
81     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
82     open(tmp_path, "wb").close()
83     with _uploads_lock:

```

```

84         _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
85     return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
86           "next_part": f"/api/upload/{upload_id}/part/0"}
87
88
89     @router.put("/api/upload/{upload_id}/part/{index}")
90     async def upload_part(upload_id: str, index: int, request: Request):
91         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
92         _, max_bytes, part_bytes, _ = _upload_cfg()
93         with _uploads_lock:
94             st = _uploads.get(upload_id)
95             if st is None:
96                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
97             if index != st["parts"]:
98                 raise HTTPException(status_code=409,
99                                   detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
100
101             received = 0
102             overflow = None
103             with open(st["tmp_path"], "ab") as fh:
104                 async for chunk in request.stream():
105                     received += len(chunk)
106                     if received > part_bytes:
107                         overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
108                         break
109                     if st["bytes"] + received > max_bytes:
110                         overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
111                         break
112                     fh.write(chunk)
113             if overflow:
114                 _abort_upload(upload_id)
115                 raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
116
117             with _uploads_lock:
118                 st["parts"] += 1
119                 st["bytes"] += received
120             return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
121
122
123     @router.post("/api/upload/{upload_id}/complete")
124     def upload_complete(upload_id: str, req: UploadCompleteRequest):
125         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
126         upload_dir, _, _, _ = _upload_cfg()
127         with _uploads_lock:
128             st = _uploads.get(upload_id)
129             if st is None:
130                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
131             if st["parts"] == 0 or st["parts"] != req.total_parts:
132                 raise HTTPException(status_code=409,
133                                   detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
134             final_path = os.path.join(upload_dir, st["filename"])
135             if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
136                 stem, ext = os.path.splitext(st["filename"])
137                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
138             os.replace(st["tmp_path"], final_path)
139             with _uploads_lock:

```

```

140         _uploads.pop(upload_id, None)
141     video_id = compute_video_id(final_path)
142     return {"path": os.path.abspath(final_path), "video_id": video_id,
143           "size_bytes": os.path.getsize(final_path),
144           "next": "POST /api/process with this path"}
145
146
147     @router.delete("/api/upload/{upload_id}")
148     def upload_abort(upload_id: str):
149         """Abort an in-flight upload and delete its partial file."""
150         with _uploads_lock:
151             known = upload_id in _uploads
152             if not known:
153                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
154             _abort_upload(upload_id)
155             return {"status": "aborted", "upload_id": upload_id}
156

```

9. user (2026-06-21T14:34:03.928Z)

```

1     # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3     > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) . **Owner:** Avi . **Created:** 2026-06-20 . **Last updated:** 2026-06-21
4     > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5     > **Tracking anchors:** $9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6     > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
screens. Engine-side changes here are **proposals to coordinate** with the active
decoupled/worker sessions ? not written unilaterally.
7     > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9     ---
10
11     ## 0. TL;DR
12     **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on
**one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment
profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the
degenerate collapse of the same storage/worker split.
13
14     The **single most important caveat:** the engine has **zero app-layer auth and CORS
`***` `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays
bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`,
and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the
tenancy boundary.**
15
16     Two honest hardware truths: (1) the current droplet is **CPU-only** ? real detection
there is **Gemini (cloud)** or **stub (CPU mock)**; **real local-GPU detection needs a GPU
host** (native WASB, in-flight). (2) The owner priority is to **stand up a GPU host ASAP** ?
provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs **in parallel**
with the UI MVP, which ships on Gemini today.
17
18     ## 1. Problem & goal
19     The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker
{infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the
FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified
backend/main.py]`. What's missing is a **console** that lets a non-CLI operator run the whole
loop on a single box, **honestly, behind a gate**.

```

```

20
21     **Headline CUJ (owner's words):** a **hosted website** where I pick an input ? **a
Google Drive file, a bucket file (`s3://`/`gs://`), or a local upload** ? it lands in the
**bucket (DO Spaces / GCS)** or locally ? I **generate highlights** on a **GPU-enabled host**
running the real e2e ? I **pick rallies, watch, and download** ? **completely from the UI.**
22
23     **Outcome ("good")** an operator signs in, picks a source, watches it index, curates
rallies, downloads a reel ? on the **live CPU droplet today** (Gemini/stub), and on a **GPU host
later** (native WASB) with **no UI fork**.
24
25     **Read to get current:** `PER_RALLY_SELECTION.md` (§3/§4/§10 contract anchors), engine
`HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built),
`VIDEO_LOCALITY_MODEL.md` (privacy/retention).
26
27     ## 2. Decisions locked
28     | # | Decision | Source / date | Implication |
29     |---|---|---|---|
30     | D1 | Reverse-proxied **single origin** ; engine stays `127.0.0.1:8000` | `[verified
main.py:638]` ; 2026-06-20 | CORS `` is moot from the browser ? the proxy is the boundary |
31     | D2 | **Edge auth** , zero engine auth code | `HOSTING_PLAN.md` ; 2026-06-20 | Caddy
basicauth OR cloudflared + Cloudflare Access |
32     | D3 | MVP drives the **in-process FastAPI worker** loop, `storage=local` | `[verified
job_worker.py:37, local.py:33]` ; 2026-06-20 | bucket worker = reserved scale path, out of MVP |
33     | D4 | Embed **RallyPicker** from `PER_RALLY_SELECTION.md` **verbatim** | PR [#11] ;
2026-06-20 | the console is a superset ; do not redesign the picker |
34     | D5 | Console lives in the khelsutra **`web/`** React scaffold | `web/README.md` ;
2026-06-20 | Cloudflare-Pages-style root, not engine static |
35     | D6 | Compute placement is a **setting** , rendered from `GET /api/capabilities` |
`[design]` ; 2026-06-20 | UI offers ONLY what the box can honestly do |
36     | D7 | **Ingest is a source menu** (local upload · bucket URI · Drive) normalizing to
one video object | `[verified backend/storage/ref.py parse_uri]` ; 2026-06-20 | Drive is a
**stub** today ? scoped honestly (§4.2) |
37     | D8 | GPU host is **provider-agnostic** (DO droplet / GCE VM / Cloud Run+GPU) ; storage
stays the bucket | Owner 2026-06-20 | compute and storage are separate planes ; no provider lock-
in |
38     | D9 | **Observability is first-class** ? structured logs on every new path + a DoD item
| Owner 2026-06-20 | see §5 ; verified during CUJ replay |
39     | D10 | Delivery = a **self-hosted webserver + UI the user spins up** (NOT a desktop
app) ; the user points at **local storage + GPU/CPU OR cloud locations** and the UI drives the
tool | Owner 2026-06-20 | **supersedes `DESKTOP_APP_PLAN.md`** ; one artifact (this appliance)
runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |
40
41     ## 3. Foundation ? what the engine gives us today `[verified this session unless
noted]`
42     - **FastAPI loop:** `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ?
`POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}`
(FileResponse) `[main.py]` . **Chunked upload exists** ? `POST /api/upload/init` · `PUT
/api/upload/{id}/part/{i}` · `POST /api/upload/{id}/complete` · `DELETE /api/upload/{id}`
`[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` **never calls them** (it only
hits `/api/process` + `/api/compile` `[verified app.js:88,373]` ). So browser upload is **client-
wiring only** , no engine change.
43     - **In-process worker:** one module-global `ThreadPoolExecutor(max_workers=1)`
`[verified job_worker.py:37]` ? strictly **one job at a time** ; the DB `jobs` table is the
clock ; `fail_state_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a
**separate** knob `indexing.ai_max_workers` (**default 5** `[verified models.py:183]` ; set to 1
to serialize) ? it is **not** already 1.
44     - **Storage:** `storage.backend` default = `local` `[verified models.py:298]` ;
`LocalStorage.fetch_to_local` is **identity** `[verified local.py:33]` , `put` no-ops on
`src==dst` ? so on a single box the worker's pull?pipeline?push **collapses to read-local/write-
local** . URI scheme `parse_uri` `[verified ref.py]` : bare `/local://` ? local ; `s3://`
(AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://`/`gs://` ; `gdrive://` ? **follow-up stub
(emulated only)** . Credentials never in config ? boto3 default chain / `~/aws` `[verified
s3.py]` .
45     - **Compute seam:** `indexing.detector_impl` ? `{auto`=WSL, `native`=Linux-native,
`stub`=CPU mock} ; default `auto` `[verified models.py:129]` , resolved in

```

```

`trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch
`feat/native-wasb-runner`:** `native_wasb_runner.py` **exists** `[verified]`;
**`fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified
fusion_hybrid.py:86]` while **`trajectory_hybrid._build_native_runner` still raises** `[verified
:57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ?
not exercisable on the droplet or in the offline suite. Treat native as **not production-
verified**.
46 - **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ?
pipeline ? pushes `outputs/<video_id>/{intervals.csv,report.json}` `[verified
worker/__main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` +
`--set indexing.skip_ai_handoff=true` + `storage.s3`.
47 - **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access
email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET
/api/health` **now EXISTS** (engine M1, [#268]) ? `{status, sport, default_segmenter}`
`[verified main.py:127, corrected 2026-06-21 ? was "does not exist"]`.
48 - **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts
the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the
full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md,
DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run
there.
49 - **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row
`get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any
**Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.
50
51 ## 4. Design / architecture
52
53 ### 4.1 Topology & request path
54 ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy /
static host** serving the `web/` SPA bundle and proxying `/api/` ? engine **FastAPI
127.0.0.1:8000** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a
GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing
site stays a **separate** systemd unit on a **separate** subdomain.
55
56 ```
57 PUBLIC INTERNET
58 ? (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app
59 ? cloudflared Tunnel + Cloudflare Access (no inbound ports) OR Caddy :443 +
basicauth
60 ?
61 ?????????????????????????????????????????????????????????????????????????????????
62 ? ONE BOX firewall BLOCKS inbound 8000 + 9000/9001/9023 ?
63 ? (2) reverse proxy / static host (Caddy/cloudflared) ?
64 ? /, /assets/* ? web/ React SPA /api/*, /api/highlights/* ? :8000 ?
65 ? ? ONE ORIGIN to the browser ? engine CORS '*' is irrelevant ?
66 ? ? ?
67 ? (3) ENGINE FastAPI (host=127.0.0.1:8000, HARD-bound [main.py:638]) ?
68 ? control: jobs table + ThreadPoolExecutor(max_workers=1) drain ?
69 ? data: uploads.py chunked upload · /api/compile · /api/highlights ?
70 ? ? ?
71 ? (4) STORAGE = LocalStorage (default) fetch=identity · put=no-op(src==dst) ?
72 ? ? ?
73 ? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet] ?
74 ? native WASB (GPU host) · auto (owner Win+WSL) ?
75 ?????????????????????????????????????????????????????????????????????????????????
76 RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO
Spaces)
77 ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3:///` ?
outputs/<vid>/ back
78 (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll
contract)
79 ```
80
81 ### 4.2 Ingest sources (the headline) ? one menu, one normalized video object
82 All sources normalize to **"a video the pipeline can read"** ? a local path (MVP) or a
`StorageRef` (scale):

```



```

83
84 | Source | Path today | Status |
85 |---|---|---|
86 | **Local upload** | SPA chunks the file ? `max_part_mb` via the existing
  `~/api/upload/init\|part\|complete` `[verified uploads.py]` ? server path ? `POST /api/process`.
  (Scale: also `StorageBackend.put` to `s3://`/`gcs://`.) | **wire client only** (endpoints exist)
  |
87 | **Bucket URI** (`s3://`, `gs://`) | **`POST /api/process` accepts the bucket URI
  directly** (resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283
  ? corrected 2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the
  **engine endpoint is NOT net-new** ? only the SPA call that POSTs the URI + polls `~/api/jobs`
  is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket
  URI 400.)* | **engine: DONE (verified); SPA ingest call: design (= B-P2)** |
88 | **Google Drive** (`gdrive://`) | `parse_uri` recognizes it, but the gdrive backend is
  an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]` | **stub ? roadmap;** MVP
  requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real |
89
90 ### 4.3 Compute placement matrix (one UI, surfaced from `~/api/capabilities`)
91 | Placement | Runs | Needs | Real/mock | Honest caveat |
92 |---|---|---|---|---|
93 | **CPU droplet ? gemini** (MVP target) | `segmenter=gemini`, in-proc worker |
  `GEMINI_API_KEY`; network; **no GPU** | **REAL** | Cloud brain: uploads ?1280px chunks to Google
  ? **must be in consent copy**; **no per-video cost ceiling** today |
94 | **CPU droplet ? stub** | trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff` |
  nothing | **MOCK** | not real tracking ? UI must label "stub (CPU mock ? not real detection)";
  demo/regression only |
95 | **CPU droplet ? motion** | `segmenter=motion` | nothing | REAL, low-quality | only no-
  key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated) |
96 | **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ?
  `NativeWasbRunner` (in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud
  Run+GPU**; `WASB_WEIGHTS` | **REAL**, on-device | partially wired (fusion builds it, trajectory
  refuses); **owner byte-parity gate**; **Cloud Run/GCE need a containerized worker ? no
  Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial** |
97 | **Owner Win+WSL ? auto** | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU
  | REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98 | **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU
  host runs `backend.worker infer` | bucket creds; a GPU host; a **net-new orchestration
  endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as
  a setting" is **design** ? selector shown **disabled** in MVP |
99
100 **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit
  for the engine's own **per-video-billing** economics (`DECOUPLED_COMPUTE` #246) ? gated on the
  **containerization feasibility check** above.
101
102 ### 4.4 Auth / exposure (non-negotiable invariant)
103 No engine auth, by design; the gate is external. **Invariant (all three must hold):**
  engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist
  `[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`**;
  set `security.allowed_video_root`; rate-limit; per-user quota.
104
105 ### 4.5 UI operator console (web/ SPA ? superset of RallyPicker)
106 - **S1 Ingest & Process** ? the source menu ($4.2) + a segmenter picker from
  `~/api/capabilities` ? `POST /api/process`.
107 - **S2 Progress** ? 3s poll of the free-text stage
  (`hashing/validating/segmenting/finalizing`), a **"one job at a time"** notice, and the restart-
  failure terminal state.
108 - **S3 Rally pick ? reel** ? **embed RallyPicker verbatim** (`PER_RALLY_SELECTION.md`):
  honor `min_shot_count`; never render motion `server/receiver/events` or a fake confidence meter.
  **Per-rally is ONE screen of the console.**
109 - **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for
  **reload-survival**.
110 - **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target
  selector shown DISABLED** ("single-box: local") until the bucket-worker bridge lands.
111
112 ### 4.6 Reuse map

```

```

113     **Reused (no re-plumb):** the full `/api` loop; the in-process worker; the chunked
upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing
`NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the
`HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. **Net-new:** Caddyfile + systemd units;
`GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the
SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker
bridge.
114
115     ## 5. Observability ? logging & monitoring `[required, first-class]`
116     Launch-readiness directive (owner 2026-06-20): **logs and monitoring taken seriously,
with a full log-analysis pass across every new code path.** The engine already uses a
centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new
path.
117
118     - **Correlation ID end-to-end.** Thread the existing `job_id` (from `/api/process`)
**plus** an `upload_id`/`request_id` through ingest ? process ? jobs ? compile ? highlights, and
into the worker for the bucket path, so **one CUJ is traceable** across UI ? FastAPI ? worker ?
GPU host. Log it on every line.
119     - **Structured logs.** New endpoints/screens emit structured (JSON-able) logs at **entry
/ result / error** ? not just "it ran". The SPA logs client-side errors + failed fetches with
the same correlation id.
120     - **Loud failures, never silent drops.** Carry the engine's ethos (#240 "surface failed
chunks loudly") + RallyPicker's `min_shot_count`-warning rule: a partial/empty reel must
**never** look complete; every dropped clip/chunk is logged + surfaced in the UI.
121     - **Health + metrics.** Ship the missing `GET /api/health` `[HOSTING_PLAN.md:42]` and a
thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a
ledger exists) for the engine-mode banner + alerting.
122     - **Aggregation for transient hosts.** A scale-to-zero Cloud Run / GPU host is
**ephemeral** ? logs must ship to a durable sink (the bucket or a log service), not die with the
instance.
123     - **Log-analysis pass before merge (DoD item).** Every PR that adds a code path includes
a check that the path logs meaningfully, and the **logs are inspected during CUJ live-replay**
($8 Layers 1?3). Ties [[feedback-launch-quality-bar]].
124
125     ## 6. Risk table
126     | Risk | Today | Mitigation |
127     |---|---|---|
128     | No app auth + CORS `` `[main.py:42]` | open if any of {localhost-bind, firewall,
gate} slips | enforce all three; **LIVE posture test** that raw `:8000` is refused from outside
|
129     | Test ports on shared droplet | MinIO `9000/9001` + GCS emu `9023` | firewall closes
them |
130     | Retention | `output/`+`uploads/` originals **never deleted** | sweep / keep-proxy-only
before any non-owner exposure |
131     | Surprise Gemini/GPU bill | **no cost ceiling/ledger** | per-video cap;
`ai_max_workers` (default 5) tunable; `max_workers=1` job serialization |
132     | Native WASB unverified | partially wired; owner-parity-gated | treat as not-
production-verified until owner byte-parity on a real GPU box |
133     | New engine endpoints | `capabilities`/`videos`/`reels`/`health` don't exist |
coordinate engine PRs with active sessions; **branch from `origin/master`, re-verify HEAD,
explicit `git add`** |
134     | Drive ingest overstated | `gdrive://` is a stub | don't present Drive as live; require
Drive?bucket pre-step until real |
135     | No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before
promising Cloud Run/GCE |
136     | Shared-checkout collisions | active worker sessions share the engine tree | the
branch-safety protocol on every engine-side PR |
137     | **Provisioning-credential leak** | a DO API token / cloud creds could leak if pasted
in chat or committed (cf. the **already-rotated Spaces key** exposed in chat,
`DECOUPLED_COMPUTE/HANDOFF.md`) | authenticate the CLI **on-box** (`doctl auth init` interactive
/ `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token,
revoke after; creds 0600, never committed; **creating paid GPU infra is a confirm-first action
every time** |
138
139     ## 7. Honest limits ? what this does NOT do

```

```

140 Does **not** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs
global). Does **not** run real WASB on the CPU droplet (Gemini or stub only). Does **not** make
the remote-GPU bucket path work (design; selector disabled). Does **not** provide in-UI source
playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does **not**
containerize (runbook; no Dockerfile `[verified]`). Does **not** verify native WASB (owner byte-
parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the
**console plumbing**, not engine rally-detection quality (a separate track).
141
142 ## 8. CUJs & live-test plan `[required]`
143 **CUJs:** **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core);
**CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters +
honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).
144
145 **Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:**
146 - **L1 ? Mock engine (hermetic, fast, default + CI).** A tiny FastAPI/MSW implementing
the **exact** contract (upload; `process` 202; `jobs` state machine incl. restart-failure;
`query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ?
CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`).
The SPA drives **all 4 CUJs in <1s**, **replayed via the Claude Preview/browser tools** (start
dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel
link via snapshot/network). Catches reload-survival (CUJ-2), the `play_segments` field-name bug
(CUJ-3), capability honesty (CUJ-4).
147 - **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main
--server` with `storage=local` + `detector_impl=stub` + `skip_ai_handoff` and assert the **mock
and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play_segments` schema,
compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock
handler **and** a real-engine contract test **in the same PR**. Runs in CI.
148 - **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1
with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety
posture test** (raw `:8000`/`:`:900x` refused from outside) as an assertion, not an assumption;
reuse the `parity.yml` dual-host pattern.
149 - **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job +
headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no
stacking**; coverage at/above the floor; **logs inspected during replay** ($5).
150 - **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU
box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/`cv2`-vs-`ffmpeg`
behavior; a multi-GB browser upload. Reported back into $9, stated plainly as **not verified**
in the offline suite.
151
152 ## 9. Deliverables & progress tracker ? **source of truth**
153 Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking; CI green gates merge; coverage ? floor; each PR
ships its tests + meaningful logs ($5).
154
155 | ID | Deliverable | Depends on | Gated? | Status | PR |
156 |----|-----|-----|-----|-----|----|
157 | T0 | This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers | ? | owner | ? |
(this PR) |
158 | T1 | Mock engine + capability fixtures + browser-replay CUJ scripts (L1) | T0 | no | ? |
? |
159 | T2 | Appliance shell: Caddyfile + systemd units; firewall; **edge auth**; rotate key
(P1) | ? | owner | ? | ? |
160 | T3 | L3 live posture test (raw `:8000`/`:`:900x` refused from outside) | T2 | no | ? | ? |
|
161 | T4 | S1 ingest (local upload wired) + segmenter picker | T1 | no | ? | ? |
162 | T5 | S2 progress (poll + one-job + restart-failure) | T4 | no | ? | ? |
163 | T6 | S3 embed **RallyPicker** + compile + download | T4 | no | ? | ? |
164 | T7 | *(engine)* `GET /api/capabilities` + `GET /api/health` + banner | T1 | coord | ? |
? |
165 | T8 | *(engine)* `GET /api/videos` + `GET /api/reels/{video_id}` (S4 reload-survival) |
? | coord | ? | ? |
166 | T9 | S5 Settings (config+capabilities; disabled compute/storage target) | T7,T8 | no |
? | ? |
167 | T10 | Observability pass: correlation id end-to-end + structured logs + metrics ($5) |
T4 | no | ? | ? |

```

```

168 | T11 | Retention sweep for `output/+`uploads/` | ? | owner | ? | ? |
169 | T12 | Ingest from bucket URI (`s3://`/`gs://`) wiring ? engine accepts URIs
(M2/[#280]) SPA ingest screen shipped (B-P2, [#56]); in-UI `gs://`reel CUJ proven live
e2e (`DRY_RUN_CUJ.md` §9) | T4 | ? | ? | engine #280 + SPA #56 |
170 | T13 | GPU host stand-up (provider-agnostic) + wire `detector_impl=native`; owner
byte-parity (P4) | branch | ? owner | ? | ? |
171 | T14 | *(reserved)* Bucket-worker bridge ? remote-GPU "compute as a setting" (P5) |
T8,T12 | ? owner | ? | ? |
172
173 Owner priority (2026-06-20): T13 (GPU host) is elevated to run in parallel from the
start ? owner provisions the host + token; the engine session lands native WASB; the UI MVP
does not block on it (ships on Gemini).
174
175 ## 10. Open questions ? owner / external
176 Blocking gates: (1) edge-auth ? Caddy basicauth (self-contained) vs
cloudflared+Cloudflare Access (matches the two-host design, no inbound ports)? (2) retention
defaults ? auto-delete originals after compile? keep proxies+intervals only? window?
(privacy/consent). (3) per-video Gemini/GPU cost ceiling ? what cap, hard-fail vs warn? (4)
GPU host choice ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU (and is a containerized worker
worth building for Cloud Run)? (5) P5 bridge ownership ? who builds the orchestration
endpoint + how it coordinates with the active worker sessions.
177 Nice-to-knows: console home (web/ vs engine static); a review-playback streaming
endpoint; per-user scoping timing; containerization vs runbook.
178
179 ## 11. Definition of done
180 - [ ] CUJ-1 works end to end on the droplet (Gemini and stub) behind the gate, via
the ingest menu.
181 - [ ] Raw `:8000`/`:900x` unreachable from outside (tested, not assumed).
182 - [ ] Local upload wired; bucket-URI ingest at least designed/tracked; Drive scoped
honestly (not faked).
183 - [ ] RallyPicker embedded with honesty rules; `min_shot_count` surfaced.
184 - [ ] `/api/capabilities` + `/api/health` + banner truthful; reload-survival via
`/api/videos` + `/api/reels`.
185 - [ ] Observability: correlation id threaded end-to-end; structured logs at every
new path; loud failures; logs inspected during CUJ replay.
186 - [ ] Retention sweep in place; `GEMINI_API_KEY` rotated; `allowed_video_root` set.
187 - [ ] CI green (L2 contract + L4 regression); coverage ? floor; §9 tracker all
?/?-resolved.
188
189 ## 12. References
190 Engine code `[verified this session]`:
`backend/main.py:12-17,42,52,307-328,331-340,342-396,638`;
`backend/api/uploads.py:64,89,123,147`; `backend/api/job_worker.py:37`;
`backend/api/static/app.js:88,344,373`;
`backend/storage/{base.py,local.py:33,ref.py,__init__.py:43}`;
`backend/config/models.py:129,183,298`;
`backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86,
gemini.py:470-485, motion.py:191-198}`; `backend/pipeline/detectors/{native_wasb_runner.py,
stub_runner.py}`; `backend/worker/__main__.py:88,98,149`; `config.json` (`default_segmenter:
gemini`). Engine docs: `HOSTING_PLAN.md:42,102-108`; `VIDEO_LOCALITY_MODEL.md`;
`DESKTOP_APP_PLAN.md`; `SETUP_NEW_MACHINE.md`; `PLATFORM_ARCHITECTURE.md`;
`DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; `UI_PROPOSAL.md:41-66`;
`PROJECT_DOC_TEMPLATE.md`. This repo: `PER_RALLY_SELECTION.md`, `REQUEST_ACCESS_FORM.md`,
`PORTABILITY.md`, `web/README.md`.
191
192 ---
193 ### Changelog
194 - 2026-06-20 ? Initial `DRAFT`. Consolidated three single-box-appliance designs
(FastAPI-native / unified-origin / decoupled-profile) into the reverse-proxied unified-origin
spine + compute-as-a-setting framing, via a multi-agent design pass; every engine-fact anchor
re-verified against source before writing (catching the native-WASB partial-wiring +
`ai_max_workers` default nuances). Folded in the owner's ingest-menu (Drive/bucket/local),
multi-provider GPU, and first-class Observability directives.
195

```

10. assistant (2026-06-21T14:34:07.145Z)

This doc is detailed. Now let me read the remaining files: PORTABILITY.md, the provision/teardown scripts, main.py (process + static serve + CORS/bind), and the config.

11. user (2026-06-21T14:34:08.131Z)

```
1      # Portability ? Cloudflare now, always-on VM if needed
2
3      > **Status:** design + proven (2026-06-20). **Decision:** keep the site/form host-
portable so
4      > moving off Cloudflare to an owner-run always-on VM ("if Cloudflare acts out") is an
**adapter
5      > swap, not a rewrite.**
6
7      ## The shape that makes it portable
8      - **Logic is runtime-agnostic.** `src/index.js` uses Web-standard `Request`/`Response`
and pure
9      helpers (`validate`, `toRow`, `isBot`, `handleRequest`) ? no host-specific calls.
10     - **All DB coupling is in adapters** (`src/store.js`): `d1Store` (Cloudflare D1) and
`sqliteStore`
11     (node:sqlite / better-sqlite3). `handleRequest(request, store)` takes the store by
injection.
12     - **One schema, one SQL.** `schema.sql` and the `INSERT` are plain SQLite, so D1 and a
VM's SQLite
13     file are byte-for-byte compatible ? zero dialect change. (Postgres later = minor
placeholder tweaks.)
14
15     ## Two hosts, same code
16     | | Cloudflare (now) | Always-on VM (fallback) |
17     |---|---|---|
18     | Entry | `src/index.js` (Worker `fetch`) | `server.js` (node:http) |
19     | Static | `env.ASSETS` | `server.js` serves `public/` |
20     | Storage | `d1Store(env.DB)` ? D1 | `sqliteStore(db)` ? `node:sqlite` file |
21     | Country | `cf-ipcountry` header | optional (GeoIP / null) |
22     | Deploy | git ? Workers Builds | `node server.js` under systemd |
23     | TLS/DNS | Cloudflare | keep Cloudflare in front, **or** Caddy/Let's Encrypt |
24
25     **Enforced, not asserted.** `node --test src/` runs BOTH paths and CI
(`.github/workflows/test.yml`,
26     Node 24) runs them on every PR, so the repo stays compatible with both hosts:
27     - `src/index.test.js` ? the Cloudflare Worker + D1 logic.
28     - `src/server.test.js` ? **boots the real `server.js`** (node:http + node:sqlite) and
round-trips it
29     over HTTP into SQLite (the same proof we run on the droplet, automated).
30
31     For a live check against a real VM: `SSH_TARGET=root@host scripts/droplet-smoke.sh` (scp
+ boot +
32     round-trip + cleanup). Verified live on the owner's droplet (Ubuntu 24.04, isolated
`/srv/khelsutra-site`).
33
34     ## Provider-portable provisioning ? destroy ? resume anywhere (scripted + TESTED)
35     Spinning up a fresh VM on **any** provider (DigitalOcean / GCE / any Ubuntu box) and
tearing it down
36     without losing data is a **first-class, scripted** flow ? so destroying a box and
resuming elsewhere is
37     routine, not risky.
38
39     - **`scripts/provision-vm.sh`** ? run from your LOCAL machine against a FRESH Ubuntu VM.
Pushes `site/`
40     via `git archive` (**private-repo-safe** ? no clone, token, or secret on the VM),
installs Node (?24,
41     so `node:sqlite` is unflagged), installs + starts a **systemd** service, opens the
firewall, and
42     **verifies** it serves (form + a real POST round-trip). Idempotent.
```

```

43     ``
44     SSH_TARGET=root@<ip> [SSH_KEY=~/.ssh/khelsutra_droplet] [PORT=8787]
[RESTORE_DB=./backup.db] \
45     site/scripts/provision-vm.sh
46     ``
47     - **`scripts/teardown-vm.sh`** ? **exports the SQLite state DB to a local timestamped
backup BEFORE**
48     anything is destroyed (registrations survive the move), stops the service, and
optionally destroys the
49     DO droplet (`DESTROY=1 DROPLET_ID=<id>`, via `doctl`).
50     ``
51     SSH_TARGET=root@<ip> [BACKUP_DIR=./vm-state-backups] [DESTROY=1 DROPLET_ID=<id>] \
52     site/scripts/teardown-vm.sh
53     ``
54     - **Clean resume on a different provider:** `teardown-vm.sh` (export) ? destroy ?
`RESTORE_DB=<backup>
55     SSH_TARGET=root@<newip> site/scripts/provision-vm.sh`. The restored DB is `scp`d in
before the service starts.
56
57     **Verified live (2026-06-20):** a fresh DO Ubuntu-24.04 droplet went bare ? operational
+ publicly
58     serving (HTTP 200) via `provision-vm.sh` (Node 24 auto-installed, systemd unit,
firewall); `teardown-vm.sh`
59     exported the state DB (12 KB); a re-provision with `RESTORE_DB` preserved the
registration (`rows:2` =
60     restored + the verify POST) ? then the throwaway droplet was destroyed. The
destroy?resume cycle is
61     reproducible across providers.
62
63     ## VM deploy runbook ("if Cloudflare acts out")
64     Node 22+ only ? **no `npm install`** (node:sqlite is built in).
65     ``
66     # on the VM:
67     git clone ? && cd khelsutra/site          # or scp the site/ folder
68     PORT=8080 DB_PATH=/var/lib/khelsutra/access.db node server.js
69     # put it under systemd (auto-restart) + a reverse proxy (Caddy/nginx) for TLS,
70     # then point DNS at the VM. ~1 hour; logic + schema unchanged.
71     ``
72
73     ## Honest notes
74     - "Cloudflare acts out" usually means **Workers/D1 pricing or limits, not DNS** ? so the
cheapest
75     move is to keep Cloudflare DNS/CDN in front and shift only the **compute** to the VM
(point a
76     route/origin at the box). Full exit (own DNS + Caddy TLS) is also supported.
77     - You can run **both** (Cloudflare primary + VM warm standby) since the schema is
identical; a
78     periodic `access_requests` export keeps them in sync.
79     - `server.js` is intentionally dependency-free; if you later want Postgres or a
framework (Hono runs
80     on Workers *and* Node), it slots in behind the same `store` + handler.
81

```

12. user (2026-06-21T14:34:09.034Z)

```

1     #!/usr/bin/env bash
2     # First-class, PROVIDER-AGNOSTIC provisioning for the KhelSutra site (Node + SQLite
host).
3     # Run from your LOCAL machine against a FRESH Ubuntu VM (DigitalOcean / GCE / any
cloud):
4     # it pushes site/ via `git archive` (PRIVATE-REPO-SAFE ? no clone, token, or secret on
the
5     # VM), installs Node, installs + starts a systemd service, opens the firewall, and
VERIFIES
6     # it is actually serving. Idempotent (safe to re-run). Pairs with teardown-vm.sh so a

```

```

7      # destroy ? resume-on-a-different-provider cycle is clean and reproducible.
8      #
9      #   SSH_TARGET=root@1.2.3.4 [SSH_KEY=~/.ssh/khelsutra_droplet] \
10     #   [REMOTE_DIR=/srv/khelsutra-site] [PORT=8787] [NODE_MAJOR=24]
[RESTORE_DB=./backup.db] \
11     #   site/scripts/provision-vm.sh
12     #
13     # NODE_MAJOR must be >= 24 so `node:sqlite` is unflagged (server.js imports it
directly).
14     set -euo pipefail
15     : "${SSH_TARGET:?set SSH_TARGET=user@host}"
16     KEY="${SSH_KEY:-$HOME/.ssh/khelsutra_droplet}"
17     DIR="${REMOTE_DIR:-/srv/khelsutra-site}"
18     PORT="${PORT:-8787}"
19     NODE_MAJOR="${NODE_MAJOR:-24}"
20     SITE="$(cd "$(dirname "$0")/.." && pwd)"          # the site/ dir (script lives in
site/scripts/)
21     REPO="$(cd "$SITE/.." && pwd)"                    # repo root (for git archive)
22     SSHO=(-i "$KEY" -o StrictHostKeyChecking=accept-new -o ConnectTimeout=10)
23
24     echo "? [1/5] waiting for SSH at $SSH_TARGET ..."
25     for _ in $(seq 1 60); do ssh "${SSHO[@]}" "$SSH_TARGET" true 2>/dev/null && break ||
sleep 5; done
26     ssh "${SSHO[@]}" "$SSH_TARGET" true              # final attempt ? fail loud if still
down
27
28     echo "? [2/5] pushing site/ via git archive (private-repo-safe; no secret on the VM)
..."
29     ssh "${SSHO[@]}" "$SSH_TARGET" "rm -rf '$DIR/site' && mkdir -p '$DIR/site'"
30     # core.autocrlf=false forces LF in the archive even when the operator runs on Windows ?
31     # autocrlf=true otherwise overrides .gitattributes eol=lf, producing CRLF on the VM and
a
32     # byte-for-byte parity drift vs Cloudflare (which serves the canonical LF blobs).
33     git -C "$REPO" -c core.autocrlf=false archive HEAD:site | ssh "${SSHO[@]}" "$SSH_TARGET"
"tar -x -C '$DIR/site'"
34
35     if [ -n "${RESTORE_DB:-}" ]; then
36         echo "? restoring state DB $RESTORE_DB ? $SSH_TARGET:$DIR/access.db (clean resume)
..."
37         scp "${SSHO[@]}" -q "$RESTORE_DB" "$SSH_TARGET:$DIR/access.db"
38     fi
39
40     echo "? [3/5] remote setup (Node $NODE_MAJOR + systemd unit + firewall) ..."
41     ssh "${SSHO[@]}" "$SSH_TARGET" "NODE_MAJOR='$NODE_MAJOR' DIR='$DIR' PORT='$PORT' bash
-s" <<'REMOTE'
42     set -euo pipefail
43     export DEBIAN_FRONTEND=noninteractive
44     have_node=0
45     if command -v node >/dev/null 2>&1; then
46         [ "$(node -p 'process.versions.node.split(".")[0]')" -ge "$NODE_MAJOR" ] &&
have_node=1
47     fi
48     if [ "$have_node" -eq 0 ]; then
49         echo " installing Node $NODE_MAJOR (NodeSource) ..."
50         apt-get update -y >/dev/null
51         apt-get install -y ca-certificates curl gnupg >/dev/null
52         curl -fsSL "https://deb.nodesource.com/setup_${NODE_MAJOR}.x" | bash - >/dev/null
53         apt-get install -y nodejs >/dev/null
54     fi
55     echo " node $(node --version)"
56     NODE_BIN="$(command -v node)"
57     cat > /etc/systemd/system/khelsutra-site.service <<UNIT
58     [Unit]
59     Description=KhelSutra site (Node + SQLite)
60     After=network.target

```

```

61
62     [Service]
63     WorkingDirectory=$DIR/site
64     Environment=PORT=$PORT
65     Environment=DB_PATH=$DIR/access.db
66     ExecStart=$NODE_BIN server.js
67     Restart=always
68
69     [Install]
70     WantedBy=multi-user.target
71     UNIT
72     systemctl daemon-reload
73     systemctl enable --now khelsutra-site
74     if command -v ufw >/dev/null 2>&1; then ufw allow "$PORT"/tcp >/dev/null 2>&1 || true;
75 fi
76
77     REMOTE
78
79     echo "? [4/5] verifying it serves (form + a real POST round-trip) ..."
80     ssh "${SSH0[@]}" "$SSH_TARGET" "PORT='$PORT' bash -s" <<'REMOTE'
81     set -e
82     for _ in $(seq 1 30); do curl -sf -o /dev/null "http://127.0.0.1:$PORT/" && break ||
83 sleep 1; done
84     form=$(curl -s "http://127.0.0.1:$PORT/" | grep -c 'id="af"' || true)
85     post=$(curl -s -X POST -H 'content-type: application/json' \
86     -d '{"email":"provision@khelsutra.guru","segment":"individual","consent":true}' \
87     "http://127.0.0.1:$PORT/api/request-access")
88     echo "  form-marker:$form api:$post"
89     { [ "$form" = "1" ] && [ "$post" = '{"ok":true}' ]; } \
90     && echo "PROVISION: OPERATIONAL" || { echo "PROVISION: FAIL"; exit 1; }
91     REMOTE
92
93     echo "? [5/5] done ? systemd 'khelsutra-site' serving on :$PORT (systemctl status
94 khelsutra-site)."
95     echo "  Public check (if firewalled open): http://<vm-ip>:$PORT/"
96

```

13. user (2026-06-21T14:34:09.143Z)

```

1      #!/usr/bin/env bash
2      # Clean teardown for the destroy ? resume-anywhere cycle. EXPORTS the SQLite state DB to
3      a
4      # local timestamped backup BEFORE you destroy the VM, so registrations survive the move
5      to a
6      # different provider. Then stops the service and (optionally) destroys the DO droplet.
7      # Resume on any fresh VM with: RESTORE_DB=<the backup> SSH_TARGET=root@NEWIP provision-
8      vm.sh
9
10     #
11     #   SSH_TARGET=root@1.2.3.4 [SSH_KEY=~/.ssh/khelsutra_droplet]
12     [REMOTE_DIR=/srv/khelsutra-site] \
13     #   [BACKUP_DIR=./vm-state-backups] [DESTROY=1 DROPLET_ID=12345678]
14     site/scripts/teardown-vm.sh
15
16     set -euo pipefail
17     : "${SSH_TARGET:?set SSH_TARGET=user@host}"
18     KEY="${SSH_KEY:-$HOME/.ssh/khelsutra_droplet}"
19     DIR="${REMOTE_DIR:-/srv/khelsutra-site}"
20     BACKUP_DIR="${BACKUP_DIR:-./vm-state-backups}"
21     SSH0=(-i "$KEY" -o StrictHostKeyChecking=accept-new -o ConnectTimeout=10)
22     mkdir -p "$BACKUP_DIR"
23
24     STAMP="$(ssh "${SSH0[@]}" "$SSH_TARGET" 'date -u +%Y%m%dT%H%M%S')"
25     OUT="$BACKUP_DIR/access-$STAMP.db"
26     echo "? [1/3] exporting state $SSH_TARGET:$DIR/access.db ? $OUT ..."
27     if ssh "${SSH0[@]}" "$SSH_TARGET" "test -f '$DIR/access.db'"; then
28         scp "${SSH0[@]}" -q "$SSH_TARGET:$DIR/access.db" "$OUT"
29     fi
30     echo "  saved $(wc -c < "$OUT" | tr -d ' ') bytes ? resume with: RESTORE_DB=$OUT ?

```



```

provision-vm.sh"
23     else
24         echo "    no access.db on the VM (nothing to export)"
25     fi
26
27     echo "? [2/3] stopping the service ..."
28     ssh "${SSH0[@]}" "$SSH_TARGET" "systemctl stop khelsutra-site 2>/dev/null || true"
29
30     if [ "${DESTROY:-}" = "1" ] && [ -n "${DROPLET_ID:-}" ]; then
31         echo "? [3/3] destroying DO droplet $DROPLET_ID (state already exported above) ..."
32         doctl compute droplet delete "$DROPLET_ID" --force
33     else
34         echo "? [3/3] VM left intact (set DESTROY=1 DROPLET_ID=<id> to destroy via doctl)."
35     fi
36     echo "? teardown done. Clean resume on any provider: RESTORE_DB=$OUT
SSH_TARGET=root@NEWIP provision-vm.sh"
37

```

14. assistant (2026-06-21T14:34:13.508Z)

Now let me look at main.py for the /api/process endpoint, static serve, CORS/bind, and the config models for security settings.

15. user (2026-06-21T14:34:14.943Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
13    logging.basicConfig(
14        level=logging.INFO,
15        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16        datefmt='%H:%M:%S'
17    )
18    logger = logging.getLogger(__name__)
19    from fastapi import FastAPI, HTTPException, Request
20    from fastapi.middleware.cors import CORSMiddleware
21    from pydantic import BaseModel
22    from typing import List, Dict, Any, Optional
23
24    from fastapi.staticfiles import StaticFiles
25    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26    from backend.pipeline.validators.base import registry
27    from backend.infrastructure.database import Database
28    from backend.pipeline.segmenters.base import segmenter_registry
29    from backend.pipeline.stitcher.compiler import VideoCompiler
30    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32    from backend.config import ( # new typed config
33        load_config as load_app_config,
34        AppConfig,
35        StitchingConfig,
36        enforce_startup_checks,
37        StartupCheckError,
38    )

```

```

39     from backend.results import Failure # standardized error/result contract
40     from backend.reporting import emit_indexer_report
41     from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
42     from backend import billing # M8: per-job cost ledger (USD internal / INR display)
43     from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
OFF)
44
45     app = FastAPI(title="Sports Highlight Indexer API")
46
47
48     def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
49         """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
50         default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
51         does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
52         src = os.environ if env is None else env
53         raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
54         origins = [o.strip() for o in raw.split(",") if o.strip()]
55         return origins or ["*"]
56
57
58     # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
browsers per the
59     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
60     # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
61     # lock it to its origin; the middleware is fixed at startup.
62     app.add_middleware(
63         CORSMiddleware,
64         allow_origins=_cors_origins_from_env(),
65         allow_credentials=False,
66         allow_methods=["*"],
67         allow_headers=["*"],
68     )
69
70     # Serves static frontend files directly (now under api/ per approved structure)

```

16. user (2026-06-21T14:34:15.573Z)

```

240         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
241         raise
242     segmentation_ran = True
243
244     if isinstance(result, Failure):
245         logger.error(f"[API] Segmenter failed: {result.message}")
246         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
247         response = {
248             "video_id": video_id,
249             "status": "failed",
250             "message": f"Segmenter '{seg_name}' failed: {result.message}",
251             "results": [r.model_dump() for r in val_results],
252             "play_segments": [],
253         }
254         return response
255
256     segments = result.value if hasattr(result, "value") else result
257     notify("finalizing")
258     _finalize_reingest(video_id, segments, play_wm, cand_wm)
259
260     response = {
261         "video_id": video_id,

```

```

262         "status": "success",
263         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
264         "results": [r.model_dump() for r in val_results],
265         "play_segments": segments,
266     }
267     return response
268 finally:
269     # Always emit the per-video observability report (next to the video, or to
270     # report.output_dir). Never raises. Surface the paths in the response.
271     paths = emit_indexer_report(
272         db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
273         val_results=val_results, val_context=val_context,
274         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
275         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
276     )
277     if paths and isinstance(response, dict):
278         response["reports"] = paths
279
280
281 @app.post("/api/process", status_code=202)
282 def process_video(req: ProcessRequest, request: Request):
283     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
284
285     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
286     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
287     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
288     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
289     """
290     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
291     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
292     # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
293     try:
294         owner_id = edge_auth.resolve_tenant(request.headers, global_config)
295     except edge_auth.EdgeAuthError as e:
296         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
297
298     allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
299     try:
300         validate_input_path(req.video_path, allowed_root=allowed_root)
301         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
302         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
303         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
304     except ValueError as e:
305         raise HTTPException(status_code=400, detail=str(e)) from e
306     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
307     # (input_ref.key ? e.g. local:/// stripped to the bare path), not the raw URI
string. A
308     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
309     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
310     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
311     # rejects a non-local URI as out-of-root.
312     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
313         raise HTTPException(status_code=404, detail=f"Video file not found at

```

```

'{{req.video_path}}')
314     if req.segmenter and not segmenter_registry.get(req.segmenter):
315         available = list(segmenter_registry.list_available().keys())
316         raise HTTPException(
317             status_code=400,
318             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
319         )
320
321     job_id = uuid.uuid4().hex
322     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
323                                 req.player_pool, req.max_frames, owner_id=owner_id):
324         raise HTTPException(status_code=500, detail="Failed to persist job record.")
325     _get_executor().submit(_drain_due_jobs)
326     return JSONResponse(
327         status_code=202,
328         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
329     )
330
331
332     @app.get("/api/jobs/{job_id}/cost")
333     def get_job_cost(job_id: str):
334         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
335         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
336         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
337         cost = db_instance.get_job_cost(job_id)
338         if cost is None:
339             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
340         fx = global_config.billing.fx_inr_per_usd
341         usd = cost.get("cost_usd")
342         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
343         cost["fx_inr_per_usd"] = fx
344         return cost
345
346
347     @app.get("/api/videos/{video_id}/cost")
348     def get_video_cost(video_id: str):
349         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
350         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
351         agg = db_instance.get_video_cost(video_id)
352         fx = global_config.billing.fx_inr_per_usd
353         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
354         agg["fx_inr_per_usd"] = fx
355         return agg
356
357
358     @app.get("/api/jobs/{job_id}")
359     def get_job(job_id: str):

```

17. assistant (2026-06-21T14:34:18.348Z)

Now let me look at the static serve setup, the bind address, and the security config model. Let me also check the i18n locale files in khelsutra and the useIngestJob hook.

18. user (2026-06-21T14:34:19.844Z)

```

70     # Serves static frontend files directly (now under api/ per approved structure)
71     os.makedirs("backend/api/static", exist_ok=True)
72     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
73
74     @app.get("/", response_class=HTMLResponse)

```

```

75     def serve_index():
76         index_path = "backend/api/static/index.html"
77         if os.path.exists(index_path):
78             with open(index_path, "r", encoding="utf-8") as f:
79                 return f.read()
80         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
81
82     # Global variables initialized at startup
83     db_instance: Optional[Database] = None
84     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
85
86     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
87     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
88     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
89     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
90     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
91     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
92     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
93     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
94         JobWaiting,
95         _arm_wake_timer,
96         _drain_due_jobs,
97         _get_executor,
98         _job_to_request,
99         _MAX_DRAIN_WAKE_SEC,
100         _maybe_record_job_cost,
101         _run_claimed_job,
102         _schedule_next_drain_if_waiting,
103     )
104
105     # Legacy thin wrapper for any remaining direct calls during transition.
106     # New code should import load_app_config / AppConfig from backend.config directly.
107     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
108         cfg = load_app_config(config_path)
109         return cfg.model_dump(mode="json")
110
111     # --- API Models ---
112     class ProcessRequest(BaseModel):
113         video_path: str
114         player_pool: Optional[List[str]] = None
115         max_frames: Optional[int] = None
116         segmenter: Optional[str] = None
117
118     class QueryRequest(BaseModel):
119         video_id: str
120         server: Optional[str] = None
121         receiver: Optional[str] = None
122         duration_min: Optional[float] = None
123         event_type: Optional[str] = None
124
125     class CompileRequest(BaseModel):
126         video_id: str
127         segment_ids: List[int]
128         output_filename: str
129
130     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
131         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
132
133         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
134         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and

```

```

135         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
136         """
137         if segments:
138             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
139         else:
140             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
141
142
143     # --- API Endpoints ---
144     @app.get("/api/config")
145     def get_config():
146         return global_config
147
148     @app.get("/api/health")
149     def health():
150         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
151         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
152         confirm the engine is reachable and which detector is wired. Never raises."""
153         sport = getattr(global_config, "sport", None)
154         indexing = getattr(global_config, "indexing", None)
155         return {
156             "status": "ok",
157             "sport": sport,
158             "default_segmenter": getattr(indexing, "default_segmenter", None),
159         }
160
161     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
162         """The full hash ? validate ? segment ? report pipeline for one video.
163
164         This is the former synchronous /api/process body, unchanged in behavior, now run on
165         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
166         the sync endpoint used to return (UI compatibility); the per-video observability
167         report is still emitted in `finally:` and grafted as response["reports"].
168
169         `progress` is an optional callable(stage: str) used to surface coarse job progress.
170         Raises on unexpected internal errors ? the caller records those as a job failure
171         (after this function has already restored the pre-run segments, see below).
172         """
173         notify = progress or (lambda stage: None)
174
175         notify("hashing")
176         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
177         # (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
178         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
179         # consumers (hash / validators / segmenter) use the resolved local path.
180         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
181         video_id = compute_video_id(local_video)
182         was_reingest = db_instance.get_video(video_id) is not None
183
184         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
185         notify("validating")
186         logger.info(f"Running validations for {req.video_path}...")
187         val_results, val_context = registry.run_all_with_context(local_video, global_config)
188         failures = [r for r in val_results if not r.passed]
189
190         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
191         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
192         seg_name = req.segmenter or default_seg

```

```
193         segmenter_actual = None
194         segmentation_ran = False
```

19. user (2026-06-21T14:34:20.536Z)

```
620         else:
621             out_dir = args.output_dir
622             out_path = os.path.join(out_dir, out_filename)
623             out_name = out_filename
624
625             os.makedirs(out_dir, exist_ok=True)
626             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
627
628             # global_config isn't initialized this early; load the typed config so the trim
629             # clip carries the same configured stitching paddings + watermark as a real
630             # highlight (watermark default-on).
631             stitching = load_app_config().stitching
632             compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
633                                     end_padding=stitching.end_padding,
634                                     watermark=stitching.watermark)
635             try:
636                 res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
637                                     args.trim_end)], out_name)
638                 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
639             except Exception as e:
640                 print(f"[ERROR] Failed to compile trimmed segment: {e}")
641             return
642
643         if args.debug_clip is not None:
644             if not args.video_path:
645                 print("[ERROR] Please provide --video-path to debug a specific clip.")
646                 return
647             run_cli_diagnose_clip(args.video_path, args.debug_clip)
648             return
649
650         # Initialize Global Config and DB
651         global global_config, db_instance
652         global_config = load_app_config() # returns typed AppConfig
653         _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
654         profile
655
656         # The CLI --output-dir overrides the configured output directory. It now lives
657         # on the typed model (OutputConfig.output_dir), so every consumer ? including
658         # /api/compile ? reads the same value instead of a write-only runtime hack.
659         global_config.output.output_dir = args.output_dir
660         os.makedirs(global_config.output.output_dir, exist_ok=True)
661
662         db_path = os.path.join(global_config.output.output_dir,
663                               global_config.output.db_name)
664         db_instance = Database(db_path)
665
666         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
667         # the executor is in-process. Mark them failed so pollers see a terminal state.
668         swept = db_instance.fail_stale_jobs()
669         if swept:
670             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
671             failed.")
672
673         # CLI processing mode (no web UI needed)
674         if args.video_path and not args.server:
675             print(f"[CLI] Processing video file: {args.video_path}")
676             # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
677             fetched to scratch.
678             # The local-existence fast-fail only applies to a local input (stat the RESOLVED
```

```
key, not the
673         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
674         storage_cfg = getattr(global_config, "storage", None)
675         try:
676             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
677         except ValueError as e:
678             print(f"[FAIL] {e}")
679         return
```

20. assistant (2026-06-21T14:34:24.118Z)

Now let me find the server bind address and the security config model, plus the i18n locales and useIngestJob hook.

21. user (2026-06-21T14:34:26.120Z)

```
756:         uvicorn.run(app, host="127.0.0.1", port=args.port)
```

22. user (2026-06-21T14:34:28.041Z)

```
280:class SecurityConfig(BaseModel):
283:    allowed_video_root: Optional[str] = None
287:    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288:    # contain upload_dir or /api/process will reject the uploaded paths.
289:    upload_dir: str = "output/uploads"
292:    max_upload_mb: int = 4096
293:    max_part_mb: int = 95
294:    allowed_upload_extensions: List[str] = ["mp4", "mov"]
```

23. user (2026-06-21T14:34:29.025Z)

No files found

24. user (2026-06-21T14:34:29.353Z)

No files found

25. assistant (2026-06-21T14:34:31.964Z)

Let me find the i18n setup (locale catalogs) and the useIngestJob hook, plus check the SecurityConfig more fully and any retention/cleanup tooling.

26. user (2026-06-21T14:34:33.097Z)

No files found

27. user (2026-06-21T14:34:33.786Z)

```
280    class SecurityConfig(BaseModel):
281        # When set, /api/process video_path must resolve under this directory (hosted/tenant
282        # mode). Default None preserves the single-user local 'any local file' workflow.
283        allowed_video_root: Optional[str] = None
284
285        # --- Chunked upload (B2, PR-2) ---
286        # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287        # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288        # contain upload_dir or /api/process will reject the uploaded paths.
289        upload_dir: str = "output/uploads"
290        # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291        # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
292        max_upload_mb: int = 4096
293        max_part_mb: int = 95
```



```

294     allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296     # --- M9 edge auth (Cloudflare Access, D9) ---
297     # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298     # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
299     # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300     # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
301     # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
302     # application's audience tag. NO secrets here ? these are public identifiers.
303     edge_auth_enabled: bool = False
304     cf_team_domain: str = ""          # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)

```

28. assistant (2026-06-21T14:34:37.040Z)

The web/ directory structure seems different than expected. Let me find where the i18n catalogs actually live and inspect the web/ layout, plus check existing retention/cleanup tooling in BHI.

29. user (2026-06-21T14:34:38.990Z)

```

web/src/api/client.test.ts
web/src/api/client.ts
web/src/api/types.ts
web/src/App.tsx
web/src/components/CompileResult.tsx
web/src/components/IngestPanel.test.tsx
web/src/components/IngestPanel.tsx
web/src/components/LanguageSwitcher.tsx
web/src/components/QueryBar.examples.test.ts
web/src/components/QueryBar.tsx
web/src/components/RallyGrid.tsx
web/src/components/SelectionFooter.tsx
web/src/hooks/useIngestJob.test.ts
web/src/hooks/useIngestJob.ts
web/src/hooks/useRallySelection.ts
web/src/i18n/hygiene.test.ts
web/src/i18n/i18n.test.tsx
web/src/i18n/index.ts
web/src/i18n/locales/en/common.json
web/src/i18n/locales/hi/common.json
web/src/i18n/locales/kn/common.json
web/src/index.css
web/src/lib/errors.test.ts
web/src/lib/errors.ts
web/src/lib/floor.test.ts
web/src/lib/floor.ts
web/src/lib/format.test.ts
web/src/lib/format.ts
web/src/lib/log.ts
web/src/lib/query.test.ts
web/src/lib/query.ts
web/src/lib/selection.test.ts
web/src/lib/selection.ts
web/src/main.tsx
web/src/test/setup.ts
=== i18n refs ===
web/src/App.tsx
web/src/components/CompileResult.tsx
web/src/components/IngestPanel.test.tsx
web/src/components/IngestPanel.tsx

```

```
web/src/components/LanguageSwitcher.tsx
web/src/components/QueryBar.tsx
web/src/components/RallyGrid.tsx
web/src/components/SelectionFooter.tsx
web/src/hooks/useIngestJob.ts
web/src/i18n/hygiene.test.ts
```

30. user (2026-06-21T14:34:39.445Z)

```
tools\cleanup_caches.py
.claude\worktrees\bhi-execpolicy\tools\cleanup_caches.py
.claude\worktrees\agent-a771ff1d85e18f2ed\tools\cleanup_caches.py
.claude\worktrees\tier1-windowing\tools\cleanup_caches.py
.claude\worktrees\w2-safeguard\tools\cleanup_caches.py
.claude\worktrees\hardchop\tools\cleanup_caches.py
.claude\worktrees\boxhill17\tools\cleanup_caches.py
.claude\worktrees\pr-stitch\tools\cleanup_caches.py
.claude\worktrees\gap-doc\tools\cleanup_caches.py
.claude\worktrees\resweep\tools\cleanup_caches.py
.claude\worktrees\recdoc\tools\cleanup_caches.py
```

31. assistant (2026-06-21T14:34:45.322Z)

Now let me read the useIngestJob hook, the i18n index, the en/common.json ingest keys, and the cleanup_caches.py tool to ground T11.

32. user (2026-06-21T14:34:46.050Z)

```
1      // Drives the front-half ingest: submit a video to POST /api/process, then poll GET
/api/jobs/{id}
2      // until it finishes, and hand the resulting video_id to the existing rally-loading
back-half.
3      //
4      // Kept i18n-FREE (returns stable IngestErrorCodes, not prose) so it unit-tests without
a
5      // translation context ? IngestPanel maps codes to localized strings. The poll loop is a
RECURSIVE
6      // setTimeout (never setInterval): each tick waits for the prior getJob to resolve, so a
slow engine
7      // can't pile up overlapping requests. It backs off, gives up after a ceiling, and is
fully
8      // cancellable on unmount / user-cancel / a superseding submit.
9      import { useCallback, useEffect, useRef, useState } from "react";
10     import { getJob, processVideo } from "../api/client";
11     import { classifyError, type IngestError } from "../lib/errors";
12     import { log, newCorrelationId } from "../lib/log";
13     import type { JobStatus } from "../api/types";
14
15     const POLL_MIN_MS = 1000;
16     const POLL_MAX_MS = 5000;
17     const POLL_BACKOFF = 1.5;
18     const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
19
20     export type IngestPhase =
21       | { status: "idle" }
22       | { status: "submitting" }
23       | { status: "polling"; progress: string | null }
24       | { status: "done"; videoId: string }
25       | { status: "error"; error: IngestError };
26
27     interface Run {
28       cancelled: boolean;
29       timer?: ReturnType<typeof setTimeout>;
30     }
```

```

31
32     export function useIngestJob(onLoaded?: (videoId: string) => void) {
33         const [phase, setPhase] = useState<IngestPhase>({ status: "idle" });
34         const runRef = useRef<Run | null>(null);
35         // Keep the latest onLoaded without making submit depend on it (a parent re-render
won't restart polls).
36         const onLoadedRef = useRef(onLoaded);
37         onLoadedRef.current = onLoaded;
38
39         const stop = useCallback(() => {
40             const run = runRef.current;
41             if (run) {
42                 run.cancelled = true;
43                 if (run.timer) clearTimeout(run.timer);
44             }
45             runRef.current = null;
46         }, []);
47
48         // Cancel any in-flight poll loop when the component unmounts (App.tsx:80-82 alive-
flag idiom).
49         useEffect(() => stop, [stop]);
50
51         const submit = useCallback(
52             (rawUri: string) => {
53                 const uri = rawUri.trim();
54                 if (!uri) {
55                     setPhase({ status: "error", error: { code: "invalidUri" } });
56                     return;
57                 }
58
59                 stop(); // supersede any prior run
60                 const run: Run = { cancelled: false };
61                 runRef.current = run;
62
63                 const cid = newCorrelationId();
64                 const startedAt = Date.now();
65                 setPhase({ status: "submitting" });
66                 log("info", "ingest.submit_start", { cid, uri });
67
68                 const fail = (error: IngestError, event: string) => {
69                     if (run.cancelled) return;
70                     setPhase({ status: "error", error });
71                     const level = error.code === "timeout" || error.code === "emptyResult" ? "warn"
: "error";
72                     log(level, event, { cid, uri, code: error.code, detail: error.detail });
73                 };
74
75                 const poll = (jobId: string, delay: number) => {
76                     run.timer = setTimeout(() => {
77                         void (async () => {
78                             if (run.cancelled) return;
79                             let job: JobStatus;
80                             try {
81                                 job = await getJob(jobId, cid);
82                             } catch (err) {
83                                 fail(classifyError(err), "ingest.poll_error");
84                                 return;
85                             }
86                             if (run.cancelled) return;
87
88                             if (job.status === "done") {
89                                 const videoId = job.video_id ?? job.result?.video_id ?? null;
90                                 // Two status layers: a `done` job with a `failed` pipeline verdict (or no
video_id) is
91                                 // a success-shaped FAILURE ? never load it, never hang.

```

```

92         if (job.result?.status === "failed" || !videoId) {
93             fail({ code: "emptyResult" }, "ingest.empty_result");
94             return;
95         }
96         setPhase({ status: "done", videoId });
97         log("info", "ingest.done", { cid, uri, video_id: videoId });
98         onLoadedRef.current?.(videoId);
99         return;
100     }
101     if (job.status === "failed") {
102         fail({ code: "engine", detail: job.error ?? "job failed" },
"ingest.job_failed");
103         return;
104     }
105     // queued | running ? reflect coarse progress (free-text phase) and keep
polling.
106     setPhase({ status: "polling", progress: job.progress ?? null });
107     if (Date.now() - startedAt > POLL_CEILING_MS) {
108         fail({ code: "timeout" }, "ingest.poll_timeout");
109         return;
110     }
111     poll(jobId, Math.min(delay * POLL_BACKOFF, POLL_MAX_MS));
112     })();
113     }, delay);
114 };
115
116 void (async () => {
117     let jobId: string;
118     try {
119         const resp = await processVideo(uri, cid);
120         jobId = resp.job_id;
121     } catch (err) {
122         fail(classifyError(err), "ingest.submit_error");
123         return;
124     }
125     if (run.cancelled) return;
126     log("info", "ingest.queued", { cid, uri, job_id: jobId });
127     setPhase({ status: "polling", progress: null });
128     poll(jobId, POLL_MIN_MS);
129     })();
130     },
131     [stop],
132 );
133
134 const cancel = useCallback(() => {
135     stop();
136     setPhase({ status: "idle" });
137     log("info", "ingest.cancel", {});
138 }, [stop]);
139
140 const reset = useCallback(() => {
141     stop();
142     setPhase({ status: "idle" });
143 }, [stop]);
144
145 const busy = phase.status === "submitting" || phase.status === "polling";
146 return { phase, submit, cancel, reset, busy };
147 }
148

```

33. user (2026-06-21T14:34:46.488Z)

```

1    // i18n scaffold (tracker B1) ? the shared localization foundation for KhelSutra Studio.
2    //
3    // Per docs/FRIENDLY_AND_MULTILINGUAL.md + the engine's I18N_PLAN.md:

```

```

4      // - keys, never prose (t("app.title"), enforced later by a no-bare-strings lint, B2);
5      // - ICU MessageFormat for interpolation/plurals (Kannada/Hindi plural rules live in
the catalog,
6      //     not the code ? replaces the SPA's hand-rolled `count > 1 ? "s" : ""`);
7      // - one negotiation order shared with the marketing site (B3);
8      // - always-fallback to `en` so a missing translation degrades to English, never a raw
key.
9      import i18n from "i18next";
10     import { initReactI18next } from "react-i18next";
11     import LanguageDetector from "i18next-browser-languagedetector";
12     import ICU from "i18next-icu";
13     import enCommon from "../locales/en/common.json";
14     import knCommon from "../locales/kn/common.json";
15     import hiCommon from "../locales/hi/common.json";
16
17     // Supported locales (D6): `en` + `kn` + `hi`. Anything else falls back to `en`.
18     export const SUPPORTED_LOCALES = ["en", "kn", "hi"] as const;
19     export type Locale = (typeof SUPPORTED_LOCALES)[number];
20     export const DEFAULT_LOCALE: Locale = "en";
21
22     // ONE persisted-preference key, shared with the marketing site's t() shim (D9 / tracker
B3), so a
23     // visitor's language choice carries across both surfaces.
24     export const LANG_STORAGE_KEY = "khelsutra.lang";
25
26     // Catalogs are co-located here (D9). `kn` + `hi` shipped (B5/B6, AI-drafted ? native
review
27     // pending). A missing key in any locale still falls back to `en` ? the by-design
invariant.
28     const resources = {
29       en: { common: enCommon },
30       kn: { common: knCommon },
31       hi: { common: hiCommon },
32     } as const;
33
34     void i18n
35       .use(ICU)
36       .use(LanguageDetector)
37       .use(initReactI18next)
38       .init({
39         resources,
40         fallbackLng: DEFAULT_LOCALE,
41         supportedLngs: [...SUPPORTED_LOCALES],
42         nonExplicitSupportedLngs: true, // en-IN -> en
43         defaultNS: "common",
44         ns: ["common"],
45         interpolation: { escapeValue: false }, // React already escapes
46         returnNull: false,
47         // Negotiation order (I18N_PLAN §2.3): persisted pref -> ?lang= -> browser -> en.
48         detection: {
49           order: ["localStorage", "querystring", "navigator"],
50           lookupLocalStorage: LANG_STORAGE_KEY,
51           lookupQuerystring: "lang",
52           caches: ["localStorage"],
53         },
54         react: { useSuspense: false },
55       });
56
57     // Keep <html lang> in sync with the active locale (screen readers + SEO use it).
Reflects the
58     // resolved content language, so an `en` fallback for an absent locale still reads as
`en`.
59     function syncDocumentLang(lng: string) {
60       if (typeof document !== "undefined") {
61         document.documentElement.lang = (i18n.resolvedLanguage || lng ||

```

```

DEFAULT_LOCALE).split("-")[0];
62     }
63   }
64   i18n.on("languageChanged", syncDocumentLang);
65   syncDocumentLang(i18n.language ?? DEFAULT_LOCALE);
66
67   export default i18n;
68

```

34. assistant (2026-06-21T14:34:54.456Z)

Now let me read the en/common.json (ingest keys) and the i18n hygiene test to understand key-parity enforcement, plus the errors.ts to understand error code classification.

35. user (2026-06-21T14:34:55.368Z)

```

1      {
2        "app": {
3          "badge": "per-rally selection · alpha",
4          "title": "Pick the rallies for your reel",
5          "intro": "Filter by <b>rally length</b> ? e.g. <i>?top 15 longest?</i> ? then fine-
6            tune by tapping cards. Everything starts <b>selected</b>; just drop the ones you don't want."
7        },
8        "lang": {
9          "label": "Language",
10         "en": "English",
11         "kn": "?????",
12         "hi": "?????"
13       },
14       "bulk": {
15         "selectAll": "Select all",
16         "clear": "Clear",
17         "invert": "Invert"
18       },
19       "mode": {
20         "demo": "Demo data ? open with <code>?video=?video_id?</code> against a running
21         engine to load a real session.",
22         "loading": "Loading rallies for <code>{videoId}</code>?",
23         "error": "Couldn't load rallies for <code>{videoId}</code>: {error}",
24         "loaded": "Loaded <b>{count}</b> rallies from this session."
25       },
26       "query": {
27         "placeholder": "e.g. over 10s · top 15 longest · shortest first",
28         "ariaLabel": "Filter rallies by length, count or order",
29         "apply": "Apply",
30         "notAvailable": "not available yet",
31         "notAvailableTitle": "?{token}? needs {kind} detection ? not available yet",
32         "unreadable": "Couldn't read that ? try ?over 10s? or ?top 15 longest?",
33         "hint": "Type over 10s for rallies longer than 10 seconds ? or tap a sample below.",
34         "try": "Sample queries:",
35         "support": "Filtering by <b>rally length</b> is fully supported. Shot-count is
36         <x>experimental</x> (under development)."
37       },
38       "grid": {
39         "empty": "No rallies to show yet.",
40         "rally": "Rally {n}",
41         "inReel": "Rally {n}, in reel",
42         "notInReel": "Rally {n}, not in reel",
43         "start": "start",
44         "long": "long",
45         "shots": "~{count} shots · experimental",
46         "shotsTitle": "Shot count is experimental (under development) and may be
47         inaccurate{conf}. Filter by rally length instead.",
48         "shotsConfidence": " ? confidence: {conf}"
49       },
50     },
51   },
52 }

```

```

46     "footer": {
47         "count": "{count, plural, one {# rally} other {# rallies}}",
48         "summaryRest": "selected · {total} total",
49         "reelName": "Reel name",
50         "reelNamePlaceholder": "my-highlights",
51         "build": "Build reel ({count})",
52         "building": "Building?",
53         "floorOne": "<b>Your only selected rally</b> has fewer than {min} shots ? the engine
would skip it, so the build would fail. Pick a longer rally.",
54         "floorAll": "<b>All {count} selected rallies</b> have fewer than {min} shots ? the
engine skips every one, so the build would fail. Pick longer rallies.",
55         "floorSomeLead": "{count, plural, one {# rally} other {# rallies}}",
56         "floorSomeRest": "with fewer than {min} shots will be skipped when building (the
engine's <code>min_shot_count</code> floor).",
57     },
58     "result": {
59         "ready": "Your reel is ready",
60         "failed": "Build failed",
61         "dismiss": "Dismiss",
62         "download": "Download {filename}"
63     },
64     "build": {
65         "noVideo": "Open with ?video=<video_id> against a running engine to build a reel ?
demo data can't be compiled."
66     },
67     "ingest": {
68         "title": "Process a video",
69         "intro": "Point KhelSutra at a video ? a bucket link or a file on the engine ? and
it finds the rallies. This can take a few minutes.",
70         "uriPlaceholder": "gs://your-bucket/match.mp4",
71         "uriAriaLabel": "Video location ? a gs:// bucket URL or a path on the engine host",
72         "hint": "Paste a gs:// bucket link, or a video path on the machine running the
engine.",
73         "submit": "Process video",
74         "submitting": "Submitting?",
75         "cancel": "Cancel",
76         "working": "Processing your video?",
77         "progress": "{progress}",
78         "retry": "Try again",
79         "failed": "Couldn't process the video: {error}",
80         "invalidUri": "Enter a video location first ? a gs:// link or a path on the
engine.",
81         "networkError": "Can't reach the engine. Is it running?",
82         "emptyResult": "Processing finished but found no rallies in this video. Try a
clearer match recording.",
83         "timeout": "This is taking longer than expected. The job may still be running ?
reload with the video link, or try again."
84     },
85     "selection": {
86         "count": "{count, plural, one {# rally selected} other {# rallies selected}}"
87     }
88 }
89

```

36. user (2026-06-21T14:34:55.709Z)

```

1     // Error helpers shared by the ingest flow and the build flow. Kept i18n-FREE: callers
map an
2     // IngestErrorCode to a localized string (so hooks stay testable without a translation
context),
3     // while the engine's own `detail` text passes through verbatim (it's the most specific
message).
4     import { ApiError } from "../api/client";
5
6     /** Pull FastAPI's {"detail": "..."} message out of an error body, else return the raw

```

```

body. */
7     export function parseDetail(body: string): string {
8         try {
9             const j = JSON.parse(body) as { detail?: unknown };
10            if (j && typeof j.detail === "string") return j.detail;
11        } catch {
12            /* not JSON ? fall through to the raw body */
13        }
14        return body;
15    }
16
17    /** Stable, localizable failure categories surfaced by the ingest flow.
18     * `engine` carries the engine's own message in `detail`; the rest are localized by
19     code. */
20
21    export type IngestErrorCode = "invalidUri" | "networkError" | "emptyResult" | "timeout"
22    | "engine";
23
24    export interface IngestError {
25        code: IngestErrorCode;
26        /** Present for `engine` ? the verbatim FastAPI detail / status; rendered as-is, not
27         translated. */
28        detail?: string;
29    }
30
31    /** Classify a thrown client error into a stable code. A network throw (ApiError with no
32     `status`)
33     * is the common "engine unreachable" case ? its own friendly code; any HTTP error
34     surfaces the
35     * engine's `detail` (e.g. an unsupported-scheme 400, a not-found 404). */
36    export function classifyError(err: unknown): IngestError {
37        if (err instanceof ApiError) {
38            if (err.status === undefined) return { code: "networkError" }; // fetch threw ?
39            engine truly unreachable
40            if (err.body) {
41                const detail = parseDetail(err.body);
42                if (detail) return { code: "engine", detail }; // a real engine message (e.g. a
43                400 unsupported-scheme)
44            }
45            // A bare 5xx with no body is almost always the engine being down / a gateway that
46            can't reach it
47            // (e.g. the dev proxy when :8000 is off) ? for a non-technical user a friendly "is
48            it running?"
49            // beats a cryptic "HTTP 500". A 4xx with no body keeps the status (it's a client-
50            side mistake).
51            if (err.status >= 500) return { code: "networkError" };
52            return { code: "engine", detail: `HTTP ${err.status}` };
53        }
54        return { code: "engine", detail: String(err) };
55    }
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

37. assistant (2026-06-21T14:34:59.792Z)

Now let me check the i18n hygiene test (key-parity enforcement), the api/types.ts, and look at the cleanup_caches.py tool plus how the engine's deploy/key handling works (GEMINI_API_KEY, google-cloud-storage).

38. user (2026-06-21T14:35:00.853Z)

```

1    // CI guardrails that keep i18n "by design" (FRIENDLY_AND_MULTILINGUAL.md §4):
2    // 1. KEY-PARITY ? every locale catalog has EXACTLY the `en` key set (the
3    extensibility proof:
4    // drop a stub locale and this lists every gap). Trivially green while only `en`
5    exists; it
6    // starts enforcing the moment a kn/hi catalog lands (B5/B6).

```



```

5      // 2. NO-BARE-STRINGS (attribute level, v1) ? user-facing component attributes
(placeholder /
6      //      aria-label / title) must route through t(), never a hardcoded literal. A curated
start per
7      //      the doc; an AST-level sweep of JSX text can extend it later.
8      import { describe, it, expect } from "vitest";
9      import { readdirSync, readFileSync } from "node:fs";
10     import { join } from "node:path";
11
12     const LOCALES_DIR = join(import.meta.dirname, "locales");
13     const SRC_DIR = join(import.meta.dirname, "..");
14
15     function flatten(obj: Record<string, unknown>, prefix = ""): string[] {
16         const keys: string[] = [];
17         for (const [k, v] of Object.entries(obj)) {
18             const key = prefix ? `${prefix}.${k}` : k;
19             if (v && typeof v === "object" && !Array.isArray(v)) keys.push(...flatten(v as
Record<string, unknown>, key));
20             else keys.push(key);
21         }
22         return keys.sort();
23     }
24
25     const loadKeys = (locale: string): string[] =>
26         flatten(JSON.parse(readFileSync(join(LOCALES_DIR, locale, "common.json"), "utf8")) as
Record<string, unknown>);
27
28     describe("i18n key-parity ? every locale has EXACTLY the en keys", () => {
29         const enKeys = loadKeys("en");
30         const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
31             .filter((d) => d.isDirectory() && d.name !== "en")
32             .map((d) => d.name);
33
34         it("en is the non-empty source catalog", () => {
35             expect(enKeys.length).toBeGreaterThan(20);
36         });
37
38         for (const loc of others) {
39             it(`${loc} matches en exactly (no missing, no extra keys)`, () => {
40                 const keys = loadKeys(loc);
41                 const missing = enKeys.filter((k) => !keys.includes(k));
42                 const extra = keys.filter((k) => !enKeys.includes(k));
43                 expect({ locale: loc, missing, extra }).toEqual({ locale: loc, missing: [], extra:
[] });
44             });
45         }
46     });
47
48     describe("no-bare-strings ? user-facing component attributes go through t()", () => {
49         const FILES = [
50             "App.tsx",
51             "components/QueryBar.tsx",
52             "components/RallyGrid.tsx",
53             "components/SelectionFooter.tsx",
54             "components/CompileResult.tsx",
55             "components/LanguageSwitcher.tsx",
56             "components/IngestPanel.tsx",
57         ];
58         // A hardcoded placeholder / aria-label / title with letters is a bare-string
regression ? it
59         // must be an expression ({t(...)}), not a quoted literal.
60         const BARE_ATTR = /\b(?:placeholder|aria-label|title)\s*=\s*"^[^"]*[A-Za-z]{2}[^"]*"$/g;
61
62         for (const f of FILES) {
63             it(`${f}: no hardcoded placeholder/aria-label/title`, () => {

```

```

64         const src = readFileSync(join(SRC_DIR, f), "utf8");
65         expect(src.match(BARE_ATTR) ?? []).toEqual([]);
66     });
67 }
68 });
69
70 describe("Indic fonts (B4) ? self-hosted OFL Noto, no runtime CDN", () => {
71     it("main.tsx bundles the OFL Noto Kannada + Devanagari fonts (self-hosted, not Google
Fonts)", () => {
72         const main = readFileSync(join(SRC_DIR, "main.tsx"), "utf8");
73         expect(main).toMatch(/@fontsource-variable\/noto-sans-kannada/);
74         expect(main).toMatch(/@fontsource-variable\/noto-sans-devanagari/);
75     });
76     it("the font stack falls back to Noto for Kannada + Devanagari glyphs", () => {
77         const css = readFileSync(join(SRC_DIR, "index.css"), "utf8");
78         expect(css).toMatch(/Noto Sans Kannada Variable/);
79         expect(css).toMatch(/Noto Sans Devanagari Variable/);
80     });
81 });
82
83 // The Studio SPA must render correctly on an AIR-GAPPED appliance (GPU/CPU/RAM, no
internet) ?
84 // every asset (fonts, styles, scripts) is bundled local, nothing pulled from a
CDN/Google Fonts.
85 describe("offline / self-contained ? no external runtime assets", () => {
86     const CDN = /googleapis|gstatic|fonts\.google|cdnjs|unpkg|jsdelivr\bcdn\.[a-z]/i;
87     const FILES = [
88         "../index.html", // web/index.html
89         "main.tsx",
90         "index.css",
91     ];
92     for (const rel of FILES) {
93         it(`${rel}: no external font/CDN reference`, () => {
94             expect(readFileSync(join(SRC_DIR, rel), "utf8")).not.toMatch(CDN);
95         });
96     }
97     it("web/index.html loads no absolute-URL <link>/<script src> (everything is
bundled/local)", () => {
98         const html = readFileSync(join(SRC_DIR, "..", "index.html"), "utf8");
99         expect(html).not.toMatch(/(?:href|src)\s*=\s*["']https?:\/\//i);
100     });
101 });
102

```

39. user (2026-06-21T14:35:01.711Z)

```

1 // Types mirror the engine's verified contract (badminton-highlight-indexer).
2 // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3 // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5 export interface RallyMetadata {
6     /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7     shot_count?: number;
8     /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9     shot_count_confidence?: string;
10     detector?: string;
11     [key: string]: unknown;
12 }
13
14 export interface RallySegment {
15     /** The integer the UI selects and passes to /api/compile as segment_ids. */
16     id: number;
17     start_time: number;
18     end_time: number;

```

```

19     duration: number;
20     /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21     server: string | null;
22     /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23     receiver: string | null;
24     metadata: RallyMetadata;
25     /** motion-only fabricated sub-events ? DO NOT surface. */
26     events?: unknown[];
27 }
28
29 export interface QueryResponse {
30     play_segments: RallySegment[];
31 }
32
33 export interface CompileResponse {
34     status: string;
35     /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36     filepath: string;
37 }
38
39 export interface EngineConfig {
40     stitching?: {
41         /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
*/
42         min_shot_count?: number;
43         [key: string]: unknown;
44     };
45     [key: string]: unknown;
46 }
47
48 /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
/api/jobs/{job_id}. */
49 export interface ProcessResponse {
50     job_id: string;
51     /** Always "queued" at submit. */
52     status: string;
53     /** Relative poll path, e.g. "/api/jobs/<id>". */
54     poll: string;
55 }
56
57 /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
58 * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
(success|failed).
59 * A `done` job whose `result.status === "failed"` (or with no video_id) is a success-
shaped
60 * FAILURE ? the UI must not treat it as a load. */
61 export interface JobStatus {
62     job_id: string;
63     status: "queued" | "running" | "done" | "failed";
64     /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
percentage. */
65     progress?: string | null;
66     /** The canonical video_id, surfaced top-level on success (also in result.video_id).
*/
67     video_id?: string | null;
68     /** Execution error (worker crash / unexpected) when status === "failed". */
69     error?: string | null;
70     result?: { status?: string; video_id?: string; message?: string } | null;
71 }
72

```

40. user (2026-06-21T14:35:02.041Z)

```

1     """"Cache-hygiene tool for the GPU highlight pipeline.

```

```

2
3 Scans the regenerable intermediates the pipeline leaves behind ? in the WSL stage
4 dir (`~/clips`) and the Windows `output/` tree ? classifies each entry as
5 **purge** (regenerable: frame caches, staged video copies, handoff clips, temp dirs)
6 or **keep** (durable: trajectory CSVs, the SQLite DB, final reels) and reports the
7 reclaimable space.
8
9 DRY-RUN BY DEFAULT ? it only *shows* what it would delete. Pass `--apply` to delete.
10
11     python -m tools.cleanup_caches                # report only (safe)
12     python -m tools.cleanup_caches --apply         # delete purge candidates
13     python -m tools.cleanup_caches --keep-video 89605e80ed01 # protect one video's
cache
14     python -m tools.cleanup_caches --older-than 7      # only purge entries >7 days old
15     python -m tools.cleanup_caches --include-wasbcache # also drop detector resume
caches
16
17 Pairs with the pipeline's own self-cleaning (indexing.{wasb,tracknet}.keep_frames=false,
18 which deletes frames after a SUCCESSFUL run); this tool reclaims what failed/old runs
19 and pre-self-cleaning videos left behind. Stdlib only; safe to run anywhere (the WSL
20 scan is skipped automatically when `wsl` is unavailable).
21 """
22 from __future__ import annotations
23
24 import argparse
25 import os
26 import shutil
27 import subprocess
28 import sys
29 from dataclasses import dataclass
30 from typing import List, Optional, Tuple
31
32 # ---- classification (pure; unit-tested) ----- #
33
34 # (category, regenerable, default_purge). default_purge=True ? deleted unless a flag
35 # protects it. wasbcache is regenerable but default_purge=False (it enables resume and
36 # is tiny) ? opt in with --include-wasbcache.
37 _KEEP = ("keep", False, False)
38
39
40 def classify(name: str, is_dir: bool, location: str) -> Tuple[str, bool, bool]:
41     """Classify a cache entry by name. Returns (category, regenerable,
default_purge)."""
42     low = name.lower()
43     if is_dir:
44         if low.endswith("_frames"):
45             return ("frame cache", True, True)
46         if low.endswith("_wasbcache"):
47             return ("detector resume cache", True, False) # opt-in via --include-
wasbcache
48         if low.startswith("chunks_") and low.endswith("_handoff"):
49             return ("ai handoff clips", True, True)
50         if low.startswith("tmp"):
51             return ("temp dir", True, True)
52         return ("other dir", False, False)
53     # files
54     if low.endswith((".csv",)):
55         return ("trajectory csv", False, False) # durable analysis output
56     if low.endswith((".db", ".sqlite", ".sqlite3")):
57         return ("database", False, False) # cached rallies ? never purge
58     if low.endswith((".log", ".sh", ".txt", ".json", ".md")):
59         return ("log/script", False, False)
60     if low.endswith((".mp4", ".avi", ".mov", ".mkv")):
61         # In the WSL stage dir, every video is a regenerable staged copy.
62         if location == "wsl":

```

```

63         return ("staged video", True, True)
64         # In output/, a 1080p/proxy is regenerable; anything else is a produced reel.
65         if "_1080p" in low or "_proxy" in low or low.endswith("_proxy.mp4"):
66             return ("proxy video", True, True)
67         return ("highlight reel", False, False)          # durable output ? keep
68     return ("other", False, False)
69
70
71 @dataclass
72 class Entry:
73     name: str
74     path: str          # display path (WSL ~/clips/<name> or windows abs)
75     size: int          # bytes
76     mtime: float      # epoch seconds (0 if unknown)
77     is_dir: bool
78     location: str      # "wsl" | "windows"
79
80     @property
81     def category(self) -> Tuple[str, bool, bool]:
82         return classify(self.name, self.is_dir, self.location)
83
84
85     def plan_deletions(entries: List[Entry], *, keep_video: Optional[str], older_than_days:
float,
86                     include_wasbcache: bool, now: float) -> Tuple[List[Entry],
List[Entry]]:
87         """Split entries into (to_delete, kept) per the flags. Pure ? no I/O."""
88         to_delete: List[Entry] = []
89         kept: List[Entry] = []
90         cutoff = older_than_days * 86400.0
91         for e in entries:
92             cat, _regen, default_purge = e.category
93             purge = default_purge or (include_wasbcache and cat == "detector resume cache")
94             if purge and keep_video and keep_video in e.name:
95                 purge = False
96             if purge and older_than_days > 0 and e.mtime > 0 and (now - e.mtime) < cutoff:
97                 purge = False
98             (to_delete if purge else kept).append(e)
99         return to_delete, kept
100
101
102 # ---- scanning (I/O) ----- #
103
104 def human(n: int) -> str:
105     f = float(n)
106     for unit in ("B", "K", "M", "G", "T"):
107         if f < 1024 or unit == "T":
108             return f"{f:.1f}{unit}" if unit != "B" else f"{int(f)}B"
109         f /= 1024
110     return f"{f:.1f}T"
111
112
113 def _wsl_lines(distro: str, cmd: str) -> List[str]:
114     """Run a bash command in WSL and return stdout lines ([]) on any failure."""
115     try:
116         res = subprocess.run(["wsl", "-d", distro, "bash", "-lc", cmd],
117                             capture_output=True, text=True, timeout=300)
118     except (OSError, subprocess.TimeoutExpired):
119         return []
120     return (res.stdout or "").splitlines()
121
122
123 def scan_wsl(distro: str, stage_dir: str) -> List[Entry]:
124     """List ~/clips entries with recursive size + mtime + type. [] if WSL absent.
125

```

```

126     Uses ``du -sb`` (recursive byte sizes, native-fast on ext4) merged with ``find
127     -printf`` (type + mtime) by basename. Deliberately avoids a ``for`` loop ? the
128     user's login shell mangles loop variables, but du/find arg-globbing is reliable.
129     """
130     if not shutil.which("wsl"):
131         return []
132     size_lines = _wsl_lines(distro, f"du -sb {stage_dir}/* {stage_dir}/.[!..]*
2>/dev/null || true")
133     meta_lines = _wsl_lines(distro, f"find {stage_dir}/ -maxdepth 1 -mindepth 1 "
134                             f"-printf '%y\\t%T@\\t%f\\n' 2>/dev/null || true")
135     meta = {} # basename -> (is_dir, mtime)
136     for line in meta_lines:
137         parts = line.split("\\t")
138         if len(parts) == 3:
139             y, mt, name = parts
140             try:
141                 meta[name] = (y == "d", float(mt))
142             except ValueError:
143                 continue
144     entries: List[Entry] = []
145     for line in size_lines:
146         if "\\t" not in line:
147             continue
148         sz, path = line.split("\\t", 1)
149         name = path.rsplit("/", 1)[-1]
150         try:
151             size = int(sz)
152         except ValueError:
153             continue
154         is_dir, mtime = meta.get(name, (False, 0.0))
155         entries.append(Entry(name=name, path=f"{stage_dir}/{name}", size=size,
156                             mtime=mtime, is_dir=is_dir, location="wsl"))
157     return entries
158
159
160     def _dir_size(path: str) -> int:
161         total = 0
162         for root, _dirs, files in os.walk(path):
163             for fn in files:
164                 try:
165                     total += os.path.getsize(os.path.join(root, fn))
166                 except OSError:
167                     pass
168         return total
169
170
171     def scan_windows(output_dir: str) -> List[Entry]:
172         """List output/ entries (top level) plus the output/wasb sub-tree."""
173         entries: List[Entry] = []
174         for base in (output_dir, os.path.join(output_dir, "wasb")):
175             if not os.path.isdir(base):
176                 continue
177             for de in os.scandir(base):
178                 # 'wasb' is scanned as its own base below ? don't also list it as a dir
179                 here.
180                 if base == output_dir and de.name == "wasb":
181                     continue
182                 try:
183                     is_dir = de.is_dir()
184                     size = _dir_size(de.path) if is_dir else de.stat().st_size
185                     mtime = de.stat().st_mtime
186                 except OSError:
187                     continue
188                 entries.append(Entry(name=de.name, path=de.path, size=size,
                                     mtime=mtime, is_dir=is_dir, location="windows"))

```

```

189         return entries
190
191
192     def _delete(e: Entry, distro: str) -> bool:
193         try:
194             if e.location == "wsl":
195                 subprocess.run(["wsl", "-d", distro, "bash", "-lc", f'rm -rf {_q(e.path)}'],
196                               capture_output=True, text=True, timeout=600, check=True)
197             elif e.is_dir:
198                 shutil.rmtree(e.path, ignore_errors=True)
199             else:
200                 os.remove(e.path)
201             return True
202         except (OSError, subprocess.SubprocessError):
203             return False
204
205
206     def _q(path: str) -> str:
207         if path.startswith("~/"):
208             return "~/'" + path[2:].replace("'", "'\\'") + "'"
209         return "'" + path.replace("'", "'\\'") + "'"
210
211
212     def _report(title: str, rows: List[Entry]) -> int:
213         print(f"\n{title}")
214         if not rows:
215             print(" (none)")
216             return 0
217         total = 0
218         for e in sorted(rows, key=lambda x: x.size, reverse=True):
219             cat = e.category[0]
220             print(f" {human(e.size):>8} {cat:<22} {e.path}")
221             total += e.size
222         print(f" {'-' * 8}")
223         print(f" {human(total):>8} TOTAL ({len(rows)} items)")
224         return total
225
226
227     def main(argv: Optional[List[str]] = None) -> int:
228         ap = argparse.ArgumentParser(description="Report/clean regenerable pipeline caches
229 (dry-run by default).")
230         ap.add_argument("--apply", action="store_true", help="actually delete (default:
231 report only)")
232         ap.add_argument("--keep-video", default=None, help="protect entries whose name
233 contains this id/substring")
234         ap.add_argument("--older-than", type=float, default=0.0, help="only purge entries
235 older than N days")
236         ap.add_argument("--include-wasbcache", action="store_true", help="also purge
237 detector resume caches")
238         ap.add_argument("--stage-dir", default="~/clips", help="WSL stage dir (default
239 ~/clips)")
240         ap.add_argument("--distro", default="Ubuntu", help="WSL distro (default Ubuntu)")
241         ap.add_argument("--output-dir", default="output", help="Windows output dir (default
242 output)")
243         ap.add_argument("--summary", action="store_true",
244                         help="print a single-line nudge (only if reclaimable >= --threshold-
245 gb); for hooks")
246         ap.add_argument("--threshold-gb", type=float, default=5.0,
247                         help="--summary stays silent below this many reclaimable GB (default
248 5)")
249         args = ap.parse_args(argv)
250
251         import time
252         now = time.time()
253         entries = scan_wsl(args.distro, args.stage_dir) + scan_windows(args.output_dir)

```

```

245     to_delete, kept = plan_deletions(
246         entries, keep_video=args.keep_video, older_than_days=args.older_than,
247         include_wasbcache=args.include_wasbcache, now=now)
248
249     if args.summary:
250         reclaim = sum(e.size for e in to_delete)
251         if reclaim >= args.threshold_gb * (1024 ** 3):
252             print(f"?? {human(reclaim)} of regenerable pipeline cache reclaimable "
253                 f"? run `python -m tools.cleanup_caches` to review, `--apply` to
delete.")
254         return 0
255
256     reclaim = _report("PURGE candidates (regenerable):", to_delete)
257     _report("KEEP (durable / protected):", kept)
258     print(f"\nReclaimable: {human(reclaim)}")
259
260     if not args.apply:
261         print("\nDRY RUN ? nothing deleted. Re-run with --apply to delete the PURGE
list.")
262         return 0
263     if not to_delete:
264         print("\nNothing to delete.")
265         return 0
266
267     print(f"\nDeleting {len(to_delete)} items...")
268     freed, failed = 0, 0
269     for e in to_delete:
270         if _delete(e, args.distro):
271             freed += e.size
272         else:
273             failed += 1
274             print(f" FAILED: {e.path}")
275     print(f"Freed ~{human(freed)}" + (f" ({failed} failed)" if failed else ""))
276     return 0
277
278
279 if __name__ == "__main__":
280     sys.exit(main())
281

```

41. assistant (2026-06-21T14:35:05.827Z)

I have a thorough picture now. Let me check a couple final things: the web/ build config (vite proxy + package.json scripts), how the engine reads GEMINI_API_KEY/google-cloud-storage for deploy, and whether there's a dev proxy already pointing /api ? :8000.

42. user (2026-06-21T14:35:08.394Z)

```

import { defineConfig } from "vitepress/config";
import react from "@vitepress/plugin-react";
import tailwindcss from "@tailwindcss/vite";

// The SPA makes SAME-ORIGIN relative `/api/*` calls. In the appliance (prod) a reverse
// proxy serves the SPA and proxies `/api` to the engine on 127.0.0.1:8000
// (docs/LOCAL_APPLIANCE_ARCHITECTURE.md). In dev, Vite proxies `/api` to the local engine
// (`python -m backend.main --server`, :8000). Override with VITE_ENGINE_ORIGIN if needed.
export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    proxy: {
      "/api": { target: "http://localhost:8000", changeOrigin: true },
    },
  },
  test: {
    // Default env is node (the pure lib/parser tests); the i18n render test opts into jsdom via

```



```

a
  // `// @vitest-environment jsdom` docblock. Match .test.tsx too so render tests are
discovered.
  environment: "node",
  include: ["src/**/*.test.{ts,tsx}"],
  // DOM-env localStorage polyfill (jsdom under Node 26 ships none); node-env files no-op.
  setupFiles: ["src/test/setup.ts"],
},
});
=== package.json scripts ===
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "typecheck": "tsc --noEmit",
    "test": "vitest run"
  },
  "deps": [
    "@fontsource-variable/noto-sans-devanagari",
    "@fontsource-variable/noto-sans-kannada",
    "i18next",
    "i18next-browser-languagedetector",
    "i18next-icu",
    "react",
    "react-dom",
    "react-i18next"
  ],
  "dev": [
    "@tailwindcss/vite",
    "@testing-library/dom",
    "@testing-library/react",
    "@types/node",
    "@types/react",
    "@types/react-dom",
    "@vitejs/plugin-react",
    "jsdom",
    "tailwindcss",
    "typescript",
    "vite",
    "vitest"
  ]
}

```

43. user (2026-06-21T14:35:09.779Z)

```

Found 7 files
backend\compute\cloud_run.py
backend\compute\__init__.py
backend\config\startup.py
backend\storage\base.py
backend\storage\gcs.py
backend\reporting\spec.yaml
backend\eval\gemini_refine.py

```

44. assistant (2026-06-21T14:35:14.811Z)

I now have everything I need to write these three tickets precisely. Let me check the IngestPanel test to mirror test conventions and verify the upload_complete returns an absolute path (which `validate\_input\_path`/`allowed\_video\_root` must accept).

45. user (2026-06-21T14:35:15.757Z)

```

1      // @vitest-environment jsdom

```

```

2      //
3      // The ingest screen end to end: type a URI ? submit ? (mocked engine) ? onLoaded gets
the video_id;
4      // and the two failure surfaces a non-technical user actually hits ? the engine's own
400 detail,
5      // and a friendly "is the engine running?" when it's unreachable. Mounts the real i18n
catalog.
6      import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
7      import { render, screen, fireEvent, cleanup, act } from "@testing-library/react";
8      import i18n from "../i18n/index";
9      import { IngestPanel } from "../IngestPanel";
10     import type { JobStatus } from "../api/types";
11
12     type FetchFn = typeof fetch;
13
14     function httpOk(status: number, json: unknown) {
15         return { ok: status >= 200 && status < 300, status, json: async () => json, text:
async () => JSON.stringify(json) };
16     }
17
18     function fetchSeq(process: unknown, jobs: JobStatus[]): FetchFn {
19         let i = 0;
20         return vi.fn(async (url: string | URL, init?: RequestInit) => {
21             const u = String(url);
22             if (u === "/api/process" && (init?.method ?? "GET") === "POST") return process;
23             if (u.startsWith("/api/jobs/")) {
24                 const body = jobs[Math.min(i, jobs.length - 1)];
25                 i += 1;
26                 return httpOk(200, body);
27             }
28             throw new Error(`unexpected fetch ${u}`);
29         }) as unknown as FetchFn;
30     }
31
32     beforeEach(async () => {
33         window.localStorage.clear();
34         await i18n.changeLanguage("en"); // guarantees init completed + a known locale for the
assertions
35     });
36
37     afterEach(() => {
38         cleanup();
39         vi.restoreAllMocks();
40         vi.unstubAllGlobals();
41         vi.useRealTimers();
42     });
43
44     const typeUri = (value: string) =>
45         fireEvent.change(screen.getByLabelText(/video location/i), { target: { value } });
46     const clickProcess = () => fireEvent.click(screen.getByRole("button", { name: /process
video/i }));
47
48     describe("IngestPanel", () => {
49         it("submits a URI, polls, and hands the video_id to onLoaded", async () => {
50             vi.useFakeTimers();
51             vi.stubGlobal(
52                 "fetch",
53                 fetchSeq(httpOk(202, { job_id: "j", status: "queued", poll: "/api/jobs/j" }), [
54                     { job_id: "j", status: "running", progress: "segmenting" },
55                     { job_id: "j", status: "done", video_id: "vidZ", result: { status: "success",
video_id: "vidZ" } } ],
56                 ),
57             );
58             const onLoaded = vi.fn();
59             render(<IngestPanel onLoaded={onLoaded} />);

```

```

60     typeUri("gs://bucket/match.mp4");
61     clickProcess();
62
63     await act(async () => {
64       await vi.advanceTimersByTimeAsync(0); // flush submit ? start polling
65     });
66     await act(async () => {
67       await vi.advanceTimersByTimeAsync(1000); // running tick
68     });
69     await act(async () => {
70       await vi.advanceTimersByTimeAsync(1500); // done tick
71     });
72     expect(onLoaded).toHaveBeenCalled("vidZ");
73   });
74
75   it("shows the engine's own 400 detail (e.g. an unsupported scheme)", async () => {
76     vi.stubGlobal("fetch", fetchSeq(httpOk(400, { detail: "unsupported storage URI
scheme: ftp://" }), []));
77     render(<IngestPanel onLoaded={vi.fn()} />);
78     typeUri("ftp://nope");
79     clickProcess();
80     expect(await screen.findByText(/unsupported storage URI scheme/i)).toBeTruthy();
81   });
82
83   it("shows a friendly, localized message when the engine is unreachable", async () => {
84     vi.stubGlobal("fetch", vi.fn(async () => {
85       throw new Error("offline");
86     }) as unknown as FetchFn);
87     render(<IngestPanel onLoaded={vi.fn()} />);
88     typeUri("gs://bucket/match.mp4");
89     clickProcess();
90     expect(await screen.findByText(/can't reach the engine/i)).toBeTruthy();
91   });
92
93   it("rejects an empty URI with a hint and never calls the engine", () => {
94     const f = vi.fn();
95     vi.stubGlobal("fetch", f);
96     render(<IngestPanel onLoaded={vi.fn()} />);
97     clickProcess();
98     expect(screen.getByTestId("ingest-error")).toBeTruthy();
99     expect(f).not.toHaveBeenCalled();
100   });
101 });
102

```

46. user (2026-06-21T14:35:16.269Z)

```

1     import { describe, it, expect, vi, afterEach } from "vitest";
2     import { listRallies, compileReel, highlightUrl, getConfig, minShotCount, ApiError }
from "./client";
3
4     type FetchFn = typeof fetch;
5
6     function mockFetch(status: number, json: unknown, text?: string): FetchFn {
7       return vi.fn(async () => ({
8         ok: status >= 200 && status < 300,
9         status,
10        json: async () => json,
11        text: async () => text ?? JSON.stringify(json),
12      } as unknown as FetchFn);
13    }
14
15    function lastCall(f: FetchFn): [string, RequestInit] {
16      const calls = (f as unknown as { mock: { calls: [string, RequestInit][] }
}).mock.calls;

```

```

17     return calls[calls.length - 1];
18 }
19
20 afterEach(() => {
21     vi.restoreAllMocks();
22     vi.unstubAllGlobals();
23 });
24
25 describe("highlightUrl", () => {
26     it("builds an encoded same-origin path", () => {
27         expect(highlightUrl("abc123", "my
reel.mp4")).toBe("/api/highlights/abc123/my%20reel.mp4");
28     });
29 });
30
31 describe("listRallies", () => {
32     it("POSTs video_id to /api/query and returns play_segments", async () => {
33         const seg = { id: 1, start_time: 0, end_time: 5, duration: 5, server: null,
receiver: null, metadata: { shot_count: 7 } };
34         const f = mockFetch(200, { play_segments: [seg] });
35         vi.stubGlobal("fetch", f);
36
37         const rallies = await listRallies("vid");
38         expect(rallies).toHaveLength(1);
39         expect(rallies[0].id).toBe(1);
40
41         const [url, init] = lastCall(f);
42         expect(url).toBe("/api/query");
43         expect(init.method).toBe("POST");
44         expect(JSON.parse(init.body as string)).toEqual({ video_id: "vid" });
45         expect((init.headers as Record<string, string>)["x-correlation-id"]).toBeTruthy();
46     });
47
48     it("returns [] when play_segments is missing", async () => {
49         vi.stubGlobal("fetch", mockFetch(200, {}));
50         expect(await listRallies("vid")).toEqual([]);

```

47. assistant (2026-06-21T14:35:20.093Z)

I have all the grounding I need. One important note: the `request()` helper in client.ts always sets `content-type: application/json` and JSON-stringifies ? the chunked-part PUT sends raw bytes, so it must bypass `request()`. Also the upload `complete` returns an absolute path, which `/api/process` then needs to accept (hosted-mode `allowed\_video\_root` must contain `upload\_dir`). I have enough to author the three tickets precisely.

48. user (2026-06-21T14:37:40.674Z)

Output does not match required schema: /tickets/1: must have required property 'effort'

49. assistant (2026-06-21T14:37:44.251Z)

The T2 ticket is missing the `effort` field. Let me resubmit with all required fields.

50. user (2026-06-21T14:39:24.067Z)

Structured output provided successfully

51. assistant (2026-06-21T14:39:44.118Z)

I've produced the three tickets for the runnable-tool North Star (shareable alpha UI, CPU/Gemini, not GPU-gated). Summary of what each covers and the load-bearing facts I verified against source:

****T4 ? In-UI local file upload (keystone, no gate, effort: medium).**** Pure client-wiring: the

engine's chunked-upload endpoints already exist (`backend/api/uploads.py`
init/part/complete/DELETE) but nothing calls them. The critical implementation trap I flagged:
`client.ts`'s `request()` helper hardcodes `content-type: application/json` + `JSON.stringify`,
so the raw-bytes part PUT must bypass it. `max\_part\_mb` comes from the init response (default
95MB, sized for edge-proxy body caps ? ties into T2), parts are strictly sequential (engine 409s
otherwise), and `complete` returns an absolute path that feeds the existing
`useIngestJob.submit` ? poll ? load ? compile ? download chain unchanged. All three i18n
catalogs must get the 11 new keys or the key-parity test (`web/src/i18n/hygiene.test.ts`) goes
red.

****T2 ? Droplet appliance shell (hosted + shareable, owner-gated, effort: large).**** Build the SPA
(`npm run build` ? `web/dist`), front it with a net-new Caddyfile (SPA at `/`, reverse-proxy
`/api/\*` ? `127.0.0.1:8000`), behind a cloudflared Tunnel + Cloudflare Access email-OTP gate on
the CPU droplet `168.144.126.176`. The non-negotiable invariant (verified `backend/main.py:756`
binds `127.0.0.1`, `:62` CORS `\*`): engine on localhost + ufw blocks 8000/9000/9001/9023 + edge
gate is sole ingress. Reuses the `provision-vm.sh`/`teardown-vm.sh` pattern (git-archive push,
systemd, ufw, verify-serves loop). Engine runs the #281-fixed drain (in master), needs
`GEMINI\_API\_KEY` (rotate the chat-shared one on-box) + `google-cloud-storage`. Cross-ticket:
`allowed\_video\_root` must contain `upload\_dir` or T4's uploads 400.

****T11 ? Retention sweep (launch blocker, owner-gated, effort: small).**** Single-tenant alpha =
global visibility of `output/` + `uploads/`. Reuses `tools/cleanup\_caches.py`'s dry-run-default
+ pure `plan\_deletions` scaffolding, but as a NEW module ? the cache classifier deliberately
KEEPS reels/.db/.csv, whereas retention must purge raw uploaded originals + orphaned
`.partial-\*`, never the SQLite DB or an in-flight upload. Reads configured
`upload\_dir`/`output\_dir` from typed config; exact window is owner-gated.

Key relevant files (all absolute): `C:/Users/avidu/Projects/khelsutra/web/src/api/client.ts`,
`.../components/IngestPanel.tsx`, `.../hooks/useIngestJob.ts`, `.../i18n/hygiene.test.ts`;
`C:/Users/avidu/Projects/badminton-highlight-indexer/backend/api/uploads.py`,
`.../backend/main.py`, `.../backend/config/models.py` (SecurityConfig at line 280),
`.../tools/cleanup\_caches.py`;
`C:/Users/avidu/Projects/khelsutra/docs/LOCAL\_APPLIANCE\_ARCHITECTURE.md`,
`.../site/scripts/provision-vm.sh`.

One repo-hygiene note: `git pull --ff-only` on khelsutra ****failed**** ? an untracked
`docs/MULTILINGUAL\_EXPANSION.md` (and the `docs/KHELSUTRA\_PRODUCT\_OVERVIEW.md` / `artifacts/` in
BHI) blocks fast-forward, so the khelsutra checkout may be behind remote. I read what's local
and did not clobber anything; worth resolving before implementation lands.